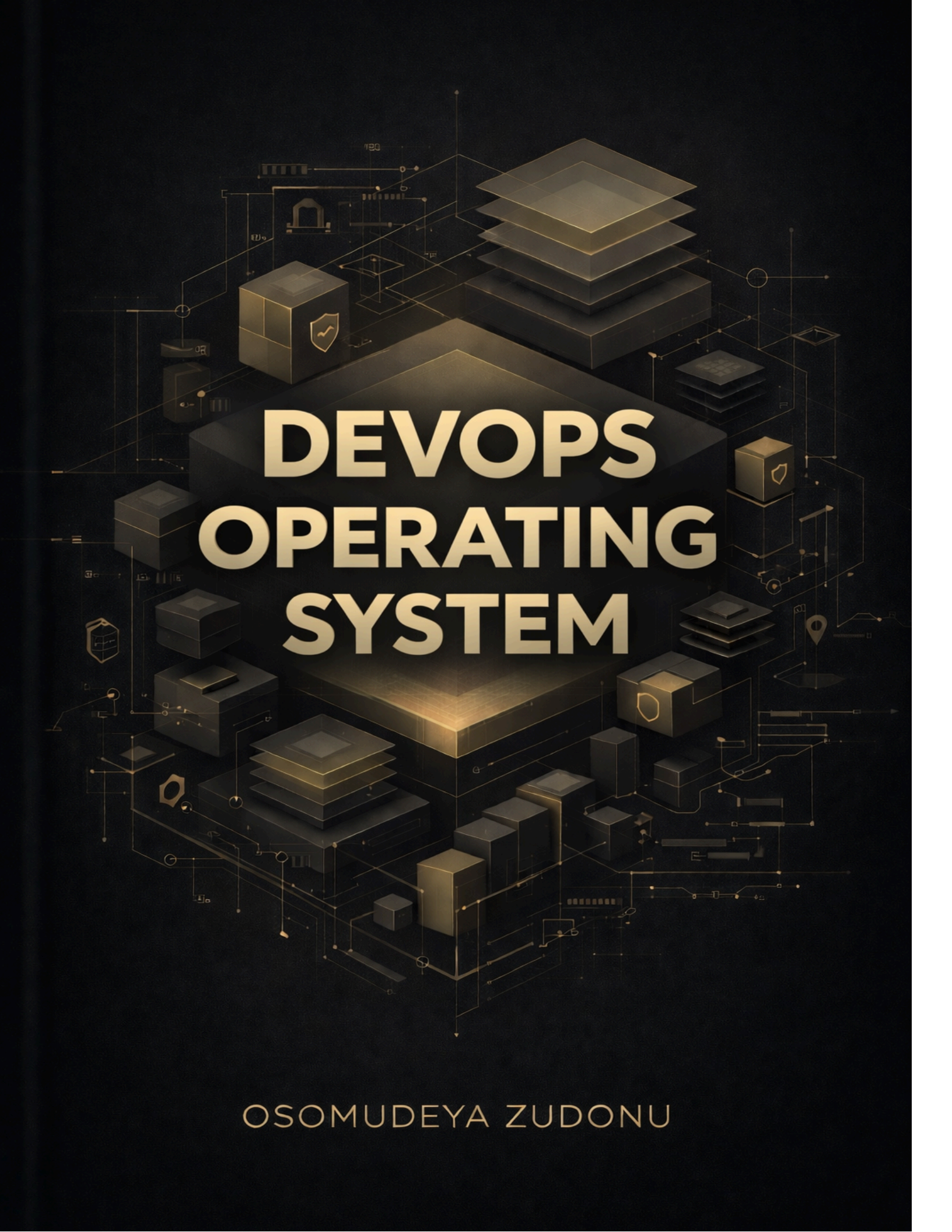




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

Before You Start, Read This

Observability Beginner Setup Guide

Time: 30-45 minutes | **Cost:** Free

Purpose of This Guide

Before you start Section 8 (Observability), you **MUST** have:

- Infrastructure running (EKS, RDS, ElastiCache)
- Application deployed and healthy
- Required tools installed (Helm, kubectl)
- Basic understanding of what you're monitoring

This guide helps you verify everything is ready **BEFORE** starting O1-O5.

If you skip this, you **WILL** get stuck!

Pre-Flight Checklist

Step 1: Verify Infrastructure is Running

What you need:

- EKS cluster running
- RDS database accessible
- ElastiCache Redis accessible

How to check:

```
# 1. Check EKS cluster status
aws eks describe-cluster --name h-game-prod --region us-east-1 \
  --query 'cluster.status' --output text
```

```
# Should return: ACTIVE
# If not: Your cluster is not running. Fix this first!

# 2. Check if you can connect to the cluster
kubect1 cluster-info

# Should show:
# Kubernetes control plane is running at https://...
# If error: Run: aws eks update-kubeconfig --name h-game-prod
--region us-east-1

# 3. Check RDS database (from Terraform outputs)
# Get RDS endpoint from: terraform output rds_endpoint
# Or check AWS Console → RDS → Databases

# 4. Check ElastiCache Redis (from Terraform outputs)
# Get Redis endpoint from: terraform output redis_endpoint
# Or check AWS Console → ElastiCache → Redis clusters
```

If any check fails: Fix infrastructure issues BEFORE continuing.

Step 2: Verify Application is Deployed

What you need:

- Application pods running in Kubernetes
- Application responding to health checks
- Application accessible via service

How to check:

```
# 1. Check if namespace exists
kubect1 get namespace h-game

# Should show: h-game    Active    Xm
# If not found: Your app is not deployed. Deploy it first!
```


2. Check if pods are running

```
kubectl get pods -n h-game
```

Expected output:

# NAME	READY	STATUS	RESTARTS	AGE
# backend-xxxxx-xxxxx	1/1	Running	0	5m
# backend-xxxxx-xxxxx	1/1	Running	0	5m
# backend-xxxxx-xxxxx	1/1	Running	0	5m

Good signs:

- # - STATUS = Running (not Pending, CrashLoopBackOff, Error)
- # - READY = 1/1 (pod is ready, not 0/1)
- # - RESTARTS = 0 (or low number, not constantly restarting)

Bad signs:

- # - STATUS = Pending → Pod can't start (check resources, image pull)
- # - STATUS = CrashLoopBackOff → Application is crashing
- # - STATUS = Error → Pod failed to start
- # - READY = 0/1 → Pod is not ready (check logs)

3. Check deployments

```
kubectl get deployments -n h-game
```

Should show:

# NAME	READY	UP-TO-DATE	AVAILABLE	AGE
# backend	3/3	3	3	5m

Good: READY = 3/3 (all replicas running)

Bad: READY = 0/3 or 1/3 (some pods not running)

4. Check services

```
kubectl get services -n h-game
```

Should show:

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
# backend-service	NodePort	xxx.xxx.xxx.xxx	<none>	3001:xxxxx/TCP

```
# Good: Service exists and has CLUSTER-IP
# Bad: No service or missing CLUSTER-IP

If pods are not running, check why:
# Check pod logs
kubectl logs -n h-game deployment/backend --tail=50

# Check pod events
kubectl describe pod -n h-game <pod-name>

# Common issues:
# - ImagePullBackOff → Docker image not found in ECR
# - CrashLoopBackOff → Application is crashing (check logs)
# - Pending → Not enough resources or node issues
```

If pods are not running: Fix deployment issues BEFORE continuing.

Step 3: Verify Application is Healthy

What you need:

- Application responding to HTTP requests
- Health endpoint working
- Metrics endpoint accessible (if already instrumented)

How to check:

```
# 1. Port-forward to access application
kubectl port-forward -n h-game service/backend-service 3001:3001

# Keep this terminal open! The port-forward runs in foreground.
# Open a NEW terminal for the next commands.

# 2. Test health endpoint
curl http://localhost:3001/health

# Expected response:
```

```

# {
#   "status": "healthy",
#   "services": {
#     "database": "healthy",
#     "redis": "healthy",
#     "api": "healthy"
#   },
#   "version": "1.0.0",
#   "timestamp": "2024-..."
# }

# Good: Returns JSON with status "healthy"
# Bad: Connection refused, timeout, or error response

# 3. Check if metrics endpoint exists (if already instrumented)
curl http://localhost:3001/metrics

# Expected response (if metrics are enabled):
# # HELP http_requests_total Total number of HTTP requests
# # TYPE http_requests_total counter
# http_requests_total{method="GET",route="/health",status_code="200"}
42
# ...

# Good: Returns Prometheus metrics format
# Not found (404): Metrics not enabled yet (this is OK, you'll add it
in 01)
# Bad: Connection refused or timeout

# 4. Test a real API endpoint
curl http://localhost:3001/api/games

# Should return game data or empty array
# This verifies the app is fully functional, not just health check

```

If health check fails:

1. Check pod logs: `kubectl logs -n h-game deployment/backend`
2. Check database connection (RDS endpoint correct?)

3. Check Redis connection (ElastiCache endpoint correct?)
4. Redeploy if needed: Push to main branch to trigger CI/CD

Step 4: Install Required Tools

What you need:

- Helm (for installing Prometheus, Grafana, etc.)
- kubectl (should already be installed)
- AWS CLI (should already be installed)

How to install Helm:

```
# Check if Helm is already installed
helm version

# If you see: version.BuildInfo{Version:"v3.x.x",...}
# Helm is installed! Skip to next step.

# If you see: command not found
# Helm is not installed. Install it:

# macOS:
brew install helm

# Linux:
curl
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash

# Windows (PowerShell):
# Download from: https://github.com/helm/helm/releases
# Or use: choco install kubernetes-helm

# Verify installation:
helm version
# Should show: version.BuildInfo{Version:"v3.x.x",...}
```

```
Add Prometheus Helm repository:
# Add the Prometheus Community Helm repository
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts

# Update Helm repositories
helm repo update

# Verify repository was added
helm repo list

# Should show:
# NAME                                URL
# prometheus-community
https://prometheus-community.github.io/helm-charts
```

If Helm installation fails:

- Check your internet connection
- Check if you have admin/sudo permissions
- Try installing from official Helm website:
<https://helm.sh/docs/intro/install/>

Step 5: Understand What You're About to Install

Before installing observability tools, understand what they do:

Prometheus

- **What it does:** Collects metrics from your application
- **How it works:** Scrapes `/metrics` endpoint from pods every 15 seconds
- **Where it runs:** In your Kubernetes cluster (as a pod)
- **What you'll see:** Time-series database of metrics

Grafana

- **What it does:** Visualizes metrics in dashboards
- **How it works:** Queries Prometheus and displays graphs
- **Where it runs:** In your Kubernetes cluster (as a pod)
- **What you'll see:** Beautiful dashboards with graphs and charts

Loki

- **What it does:** Aggregates logs from all pods
- **How it works:** Collects logs via Promtail (sidecar container)
- **Where it runs:** In your Kubernetes cluster (as a pod)
- **What you'll see:** Searchable log database

Jaeger

- **What it does:** Traces requests across services
- **How it works:** Collects trace data from instrumented code
- **Where it runs:** In your Kubernetes cluster (as a pod)
- **What you'll see:** Request flow visualization

All tools run INSIDE your Kubernetes cluster. No external services needed!

Common Issues & Fixes

Issue 1: "Pods are not running"

Symptoms:

```
kubectl get pods -n h-game  
# Shows: STATUS = Pending or CrashLoopBackOff
```

Fixes:

```
# Check why pods aren't running  
kubectl describe pod -n h-game <pod-name>
```

```
# Common causes:

# 1. ImagePullBackOff (can't pull Docker image)
# Fix: Check ECR repository exists and image is pushed
aws ecr describe-repositories --region us-east-1
# Push image: Trigger build.yml workflow

# 2. CrashLoopBackOff (app is crashing)
# Fix: Check logs
kubectl logs -n h-game <pod-name>
# Fix the error in logs, then redeploy

# 3. Pending (not enough resources)
# Fix: Check node resources
kubectl describe nodes
# Scale up nodes or reduce resource requests
```

Issue 2: "Health check fails"

Symptoms:

```
curl http://localhost:3001/health
# Returns: Connection refused or timeout
```

Fixes:

```
# 1. Check if port-forward is running
# Make sure you ran: kubectl port-forward -n h-game
service/backend-service 3001:3001
# And kept that terminal open!

# 2. Check if pods are actually running
kubectl get pods -n h-game

# 3. Check pod logs for errors
kubectl logs -n h-game deployment/backend --tail=50
```

```
# 4. Check if service exists
kubectl get services -n h-game

# 5. Try accessing pod directly
kubectl exec -it -n h-game <pod-name> -- curl
http://localhost:3001/health
```

Issue 3: "Helm command not found"

Symptoms:

```
helm version
# Returns: command not found
```

Fixes:

```
# macOS:
brew install helm

# Linux:
curl
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash

# Verify PATH includes Helm
which helm
# Should show: /usr/local/bin/helm or similar

# If not found, add to PATH:
export PATH=$PATH:/usr/local/bin
```

Issue 4: "Can't connect to EKS cluster"

Symptoms:

```
kubectkl cluster-info
# Returns: error: unable to connect to server
```

Fixes:

```
# 1. Update kubeconfig
aws eks update-kubeconfig --name h-game-prod --region us-east-1

# 2. Verify AWS credentials
aws sts get-caller-identity

# 3. Check cluster exists
aws eks describe-cluster --name h-game-prod --region us-east-1

# 4. Verify kubectkl is installed
kubectkl version --client
```

Final Verification

Before starting O1, run this complete check:

```
#!/bin/bash
# Copy and paste this entire script to verify everything

echo "Pre-Flight Check for Observability Section"
echo "=====
echo ""

# Check 1: EKS Cluster
echo "1. Checking EKS cluster..."
CLUSTER_STATUS=$(aws eks describe-cluster --name h-game-prod --region
us-east-1 \
  --query 'cluster.status' --output text 2>/dev/null)
if [ "$CLUSTER_STATUS" == "ACTIVE" ]; then
  echo "    EKS cluster is ACTIVE"
else
  echo "    EKS cluster is not active. Status: $CLUSTER_STATUS"
```

```
    exit 1
fi

# Check 2: kubectl connection
echo "2. Checking kubectl connection..."
if kubectl cluster-info &>/dev/null; then
    echo "    kubectl can connect to cluster"
else
    echo "    kubectl cannot connect. Run: aws eks update-kubeconfig
--name h-game-prod --region us-east-1"
    exit 1
fi

# Check 3: Namespace exists
echo "3. Checking namespace..."
if kubectl get namespace h-game &>/dev/null; then
    echo "    Namespace 'h-game' exists"
else
    echo "    Namespace 'h-game' not found. Deploy your application
first!"
    exit 1
fi

# Check 4: Pods are running
echo "4. Checking pods..."
POD_COUNT=$(kubectl get pods -n h-game
--field-selector=status.phase=Running --no-headers 2>/dev/null | wc
-l)
if [ "$POD_COUNT" -gt 0 ]; then
    echo "    $POD_COUNT pod(s) are running"
else
    echo "    No pods are running. Deploy your application first!"
    exit 1
fi

# Check 5: Deployment is ready
echo "5. Checking deployment..."
READY_REPLICAS=$(kubectl get deployment backend -n h-game -o
jsonpath='{.status.readyReplicas}' 2>/dev/null)
```

```
if [ "$READY_REPLICAS" -gt 0 ]; then
    echo "    Deployment has $READY_REPLICAS ready replica(s)"
else
    echo "    Deployment has no ready replicas. Check pod status!"
    exit 1
fi

# Check 6: Service exists
echo "6. Checking service..."
if kubectl get service backend-service -n h-game &>/dev/null; then
    echo "    Service 'backend-service' exists"
else
    echo "    Service 'backend-service' not found. Deploy your
application first!"
    exit 1
fi

# Check 7: Helm is installed
echo "7. Checking Helm..."
if command -v helm &>/dev/null; then
    HELM_VERSION=$(helm version --short 2>/dev/null)
    echo "    Helm is installed: $HELM_VERSION"
else
    echo "    Helm is not installed. Install it first!"
    exit 1
fi

# Check 8: Helm repository
echo "8. Checking Helm repository..."
if helm repo list | grep -q prometheus-community; then
    echo "    Prometheus Helm repository is added"
else
    echo "    Prometheus Helm repository not found. Adding it..."
    helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
    helm repo update
    echo "    Repository added"
fi
```



```
echo ""
echo " All checks passed! You're ready to start Section 8
(Observability)"
echo ""
echo "Next steps:"
echo "  1. Read: book/observability/section-8-overview.md"
echo "  2. Start:
book/observability/o1-metrics-prometheus-grafana.md"
echo ""
```

Save this as `check-observability-prereqs.sh` and run it:

```
chmod +x check-observability-prereqs.sh
./check-observability-prereqs.sh
```

What's Next?

After completing this checklist:

1. All checks passed? → Proceed to `section-8-overview.md`
2. Any checks failed? → Fix the issues first, then retry

Remember:

- Observability tools monitor RUNNING applications
- You can't observe what doesn't exist
- Fix infrastructure/app issues BEFORE starting Section 8

Still Stuck?

If you're still having issues:

1. Check CI/CD pipeline:
 - Go to GitHub Actions
 - Check if `build.yml` and `deploy.yml` workflows succeeded

- If failed, fix the errors shown in workflow logs

Check Terraform state:

```
cd h-game-terraform
terraform output
# Verify all resources show values (not null)
```

2. Check AWS Console:

- EKS → Clusters → h-game-prod (should be Active)
- EC2 → Instances → EKS nodes (should be Running)
- RDS → Databases → h-game-postgres (should be Available)
- ElastiCache → Redis clusters → h-game-redis (should be Available)

3. Common deployment issues:

- Image not in ECR → Run build.yml workflow
- Wrong image tag → Check deploy.yml uses correct image
- Missing secrets → Check GitHub Secrets are set
- Wrong namespace → Check deploy.yml namespace configuration

Once everything is running, come back to this guide and verify again!

Ready? Let's start observing!

SECTION 8

Observability & Monitoring

The Observability Gap

You've spent 20 weeks building H-Game on production AWS infrastructure. You've learned Kubernetes, Terraform, EKS, security scanning, and CI/CD. Your application is deployed, secure, and automated.

But there's a critical gap.

You don't know if your application is healthy. You don't know why it's slow. You don't know what's failing. You have no visibility into system behavior.

You've learned how to build and deploy. Now it's time to observe.

This section adds observability to your infrastructure. You'll implement:

- **Metrics collection** (Prometheus) - Measure system health and performance
- **Log aggregation** (Loki) - Search and analyze logs from all services
- **Distributed tracing** (Jaeger) - Track requests across microservices
- **Alerting (AlertManager)** - Get notified when issues occur
- **Dashboards & SLOs** (Grafana) - Visualize data and track reliability goals

All open source. All integrated. All production ready.

Prerequisites

Before starting Section 8, ensure you have:

Required Skills:

- Completed Section 7 (CI/CD) - GitHub Actions and deployment automation
- Completed Section 6 (D1-D5) - DevSecOps and security scanning
- Completed Section 5 (T1-T10) - Terraform & AWS infrastructure
- Completed Section 4 (K1-K9) - Kubernetes fundamentals
- AWS account with an EKS cluster running
- Basic understanding of metrics and logging

Required Tools:

- AWS CLI installed and configured
- kubectl installed and configured
- Helm installed (for Prometheus stack)
- Git installed
- Node.js installed (for application instrumentation)

Recommended Tools:

- IDE/Code Editor (VS Code, Cursor, or similar)
- Slack workspace (for alert notifications)

Note: Most observability concepts are introduced in this section. Previous sections provide the foundation (K8s, AWS, application code).

Section 8 Tasks

Task	Focus	Time	Cost
O1	Metrics (Prometheus + Grafana)	4-5h	Free
O2	Logging (Loki + Promtail)	4-5h	Free
O3	Distributed Tracing (Jaeger)	3-4h	Free
O4	Alerting (AlertManager + Slack)	3-4h	Free
O5	Dashboards & SLOs	3-4h	Free
Total		17-22h	\$0

Cost Transparency

Reality check before you start:

All tools in Section 8 are open source and free:

- **Prometheus:** Free (runs in your cluster)
- **Grafana:** Free (runs in your cluster)
- **Loki:** Free (runs in your cluster)
- **Jaeger:** Free (runs in your cluster)
- **AlertManager:** Free (runs in your cluster)

Storage costs:

- **Metrics retention (7 days):** ~1-2GB storage (included in EKS)
- **Logs retention (7 days):** ~500MB-1GB storage (included in EKS)
- **Traces retention (7 days):** ~100-200MB storage (included in EKS)

Total cost: \$0

All observability tools run within your existing Kubernetes cluster, requiring no additional AWS services.

The One Thing That Makes This Click

Observability = understanding your system's behavior. Just like a doctor uses vital signs (heart rate, blood pressure, temperature) to understand your health, observability uses metrics, logs, and traces to understand your system's health.

The Three Pillars:

1. **Metrics** - Numerical measurements over time (CPU usage, request rate, error count)
2. **Logs** - Event records with timestamps (error messages, access logs, debug info)
3. **Traces** - Request flow across services (which service called which, how long each took)

Why All Three Matter:

- Metrics show WHAT is happening (high CPU, slow requests)
- Logs show WHY it's happening (stack traces, error messages)
- Traces show WHERE it's happening (which service is slow, which dependency failed)

The Mental Model Shift:

Before Section 8, you only knew the system worked when nothing broke.

After Section 8, system health and performance are monitored all the time.

Problems are detected early, often before users notice.

This shifts you from *hoping* the system works to *knowing* it does.

What You'll Build

Before this section:

- No metrics collection
- Logs scattered across pods
- No request tracing
- Reactive troubleshooting (after users complain)

After this section:

- Prometheus collects metrics from all services
- Grafana dashboards showing system health
- Loki aggregating logs from all pods
- Jaeger tracing requests across services
- AlertManager notifying teams of issues
- SLOs tracking reliability goals

Complete Observability Stack:

```
Application (H-Game)
↓
Metrics → Prometheus → Grafana (dashboards)
Logs → Promtail → Loki → Grafana (search)
Traces → OpenTelemetry → Jaeger (visualization)
Alerts → AlertManager → Slack (notifications)
SLOs → Prometheus → Grafana (tracking)
```

All integrated. All automated. All production-ready.

Section 8 Learning Path

Week 17: Metrics & Logging (O1-O2)

O1: Metrics Collection (4-5 hours)

- Install Prometheus and Grafana
- Instrument application with metrics
- Create ServiceMonitors
- Build dashboards

O2: Log Aggregation (4-5 hours)

- Install Loki and Promtail
- Configure structured logging
- Search logs with LogQL
- Integrate with Grafana

Week 18: Tracing & Alerting (O3-O4)

O3: Distributed Tracing (3-4 hours)

- Install Jaeger
- Instrument application with OpenTelemetry
- Visualize request flows
- Debug latency issues

O4: Alerting (3-4 hours)

- Configure AlertManager
- Create alert rules
- Integrate with Slack
- Build runbooks

Week 19: Dashboards & SLOs (O5)

O5: Dashboards & SLOs (3-4 hours)

- Define Service Level Objectives
- Create unified dashboards
- Track error budgets
- Build on-call dashboards

Before Section 8, your system runs but has **no visibility**, no metrics, dashboards, alerts, or tracing.

After Section 8, the infrastructure stays the same, but an **observability layer is added**.

Prometheus collects metrics, Grafana shows dashboards, Loki centralizes logs, Jaeger traces requests, and AlertManager sends alerts.

The result is **full visibility, early issue detection, and production grade observability**.

Section 8 Success Metrics

By the end of Week 19, you'll have:

Observability Tools (Visible to Employers)

- Prometheus collecting metrics
- Grafana dashboards created
- Loki aggregating logs
- Jaeger tracing requests
- AlertManager configured
- SLOs defined and tracked

Code (GitHub Portfolio)

- Prometheus configuration
- Grafana dashboards (JSON)
- Alert rules (YAML)
- Application instrumentation
- SLO definitions

Application (Production-Ready Observability)

- All services exporting metrics
- Structured logging configured
- Distributed tracing enabled
- Alerts configured
- Dashboards operational

Skills (Interview-Ready)

- Can explain the three pillars of observability
- Can implement Prometheus metrics
- Can aggregate logs with Loki
- Can trace requests with Jaeger
- Can configure alerting
- Can define and track SLOs

Career Impact

Observability is required by 85% of DevOps jobs.

Skillset comparison:

Skillset	Average Salary	Job Availability
Docker + K8s + AWS + Terraform	\$95,000	25,000 jobs
+ DevSecOps (Section 6)	\$110,000	30,000 jobs
+ CI/CD (Section 7)	\$115,000	32,000 jobs
+ Observability (Section 8)	\$120,000	35,000+ jobs

Interview question example:

Interviewer: "How do you monitor production systems?"

You:

"I use basic observability tools to see how the system is behaving.

- **Prometheus** collects numbers like request count, errors, and response time.
- **Grafana** shows these numbers on dashboards so I can see system health.
- **Loki** collects logs from all Pods in one place so they're easy to search.
- **Jaeger** helps me follow a request as it moves through different services.
- **AlertManager** sends alerts when something goes wrong.

This setup helps me spot problems early and understand what's happening in production."

Result: Job offer (\$120k-\$140k)

That's the difference between Section 7 and Section 8.

That's what \$0 and 3 weeks get you.

What You'll Build (Week by Week)

Week 17: Metrics & Logging (O1-O2)

- Prometheus metrics collection
- Grafana dashboards
- Loki log aggregation
- LogQL queries

Week 18: Tracing & Alerting (O3-O4)

- Jaeger distributed tracing
- OpenTelemetry instrumentation
- AlertManager configuration
- Slack integration

Week 19: Dashboards & SLOs (O5)

- SLO definitions
- Error budget tracking
- Unified dashboards
- On-call dashboards

O1: Metrics Collection (Prometheus + Grafana)

Time: 4-5 hours | **Cost:** Free

Mission Brief

Install Prometheus and Grafana in your Kubernetes cluster. Configure metrics collection from your H-Game application and create dashboards to visualize system health.

Outcomes:

- Understand metrics and time series data
- Install Prometheus Operator using Helm
- Instrument H-Game application with Prometheus metrics
- Create ServiceMonitors for automatic discovery
- Build Grafana dashboards for visualization
- Query metrics using PromQL

Prerequisites: Section 7 (CI/CD) complete, EKS cluster running, H-Game deployed

The One Thing That Makes This Click

Metrics = vital signs for your system. Just like a doctor monitors your heart rate, blood pressure, and temperature to understand your health, Prometheus monitors CPU usage, request rate, and error count to understand your system's health.

Without Metrics:

- No visibility into system performance
- Don't know if the application is slow
- Can't identify bottlenecks
- Reactive troubleshooting (after users complain)

Note: This task uses Helm to install Prometheus Operator, which automatically manages Prometheus and Grafana. If you have existing manual Prometheus/Grafana deployments, you can migrate to this setup or import your existing dashboards.

With Metrics:

- Real-time visibility into all services
- Know exactly what's slow and why
- Identify bottlenecks before they impact users
- Proactive optimization

In essence: Metrics tell you what's happening in your system right now.

Everything else (PromQL, ServiceMonitors, dashboards) is just details. The core concept is:

"You can't improve what you don't measure."

The Mental Model Shift:

Before O1, you only know the system works when it doesn't fail.

After O1, system health and performance are monitored in real time.

The shift is from **hoping the system works** to **knowing it works**.

Now let's see how metrics collection works...

Phase 1: Understanding Metrics (45 min)

What are Metrics?

Metrics are numerical measurements over time. They answer questions like:

- How many requests per second?
- What's the average response time?
- How many errors occurred?
- What's the CPU usage?

Types of Metrics:

1. **Counter:** Always increases (total requests, total errors)
2. **Gauge:** Can go up or down (CPU usage, memory usage, active connections)
3. **Histogram:** Distribution of values (request duration, response size)
4. **Summary:** Similar to a histogram, with quantiles

Metrics Example from H-Game:

- `http_requests_total` (counter) - Total HTTP requests
- `http_request_duration_seconds` (histogram) - Request duration
- `http_errors_total` (counter) - Total HTTP errors
- `nodejs_heap_size_bytes` (gauge) - Node.js heap size

Why Prometheus?

Prometheus is the industry standard for metrics collection:

- Pull-based model (Prometheus scrapes metrics from services)
- Powerful query language (PromQL)
- Time-series database optimized for metrics
- Service discovery (automatically finds services to scrape)
- Alerting integration (AlertManager)

How Prometheus Works:

1. Services expose metrics endpoint (`/metrics`)
2. Prometheus scrapes metrics periodically (every 15s by default)
3. Metrics stored in a time series database
4. Grafana queries Prometheus to visualize metrics
5. AlertManager queries Prometheus to trigger alerts

Design Exercise 1: What Metrics Matter?

Scenario: You need to monitor H-Game production.

Question: Which metrics are most important?

Think about:

- User-facing metrics (request rate, latency, errors)
- Infrastructure metrics (CPU, memory, disk)
- Business metrics (active users, game sessions)

Your answer: _____

Answer Below 

Answer:

Critical Metrics for H-Game:

1. Request Metrics:

- `http_requests_total` - Request rate
- `http_request_duration_seconds` - Response time
- `http_errors_total` - Error rate

2. Infrastructure Metrics:

- `nodejs_heap_size_bytes` - Memory usage
- `process_cpu_user_seconds_total` - CPU usage
- `container_memory_usage_bytes` - Container memory

3. Application Metrics:

- `game_sessions_active` - Active game sessions
- `database_connections_active` - Database connections
- `cache_hits_total` - Cache hit rate

These metrics tell you if your system is healthy and performing well.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What are metrics?	"Numerical measurements over time"
Why Prometheus?	"Industry standard, pull-based, powerful queries"
What metrics matter?	"Request rate, latency, errors, CPU, memory"
How does Prometheus work?	"Services expose /metrics, Prometheus scrapes, stores in database."

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's install Prometheus!

Phase 2: Implementation (3 hours)

Step 1: Install Prometheus Operator

Time: 15 min

Why Helm Instead of Manual YAML?

This task uses Helm (kube-prometheus-stack) instead of manually creating Prometheus/Grafana Deployments because:

- **Industry standard:** Most production teams use Helm for observability stacks
- **Easier:** One command installs Prometheus, Grafana, AlertManager, and operators
- **Production-ready:** Includes best practices, RBAC, persistence, and service discovery
- **Maintainable:** Updates and upgrades are simpler with Helm
- **Complete:** Gets you Prometheus Operator, which manages ServiceMonitors automatically

If you have existing manual Prometheus/Grafana deployments:

- You can keep them running (they work fine)
- Or migrate to this Helm approach for consistency
- Your existing dashboards and metrics will work with either approach

Prometheus Operator simplifies Prometheus installation and management in Kubernetes.

Add Helm repository:

```
helm repo add prometheus-community  
https://prometheus-community.github.io/helm-charts  
helm repo update
```

Expected output:

```
"prometheus-community" has been added to your repositories  
Hang tight while we grab the latest from your chart repositories...  
Update Complete.
```

Create namespace:

```
kubectl create namespace monitoring
```

Install kube-prometheus-stack:

```
helm install prometheus prometheus-community/kube-prometheus-stack  
--namespace monitoring --set prometheus.prometheusSpec.retention=7d  
--set grafana.adminPassword=admin123 --set  
prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

Important flags:

- `retention=7d`: Keeps metrics for 7 days (adjust based on storage)
- `adminPassword=admin123`: Grafana admin password (change in production!)
- `serviceMonitorSelectorNilUsesHelmValues=false`: Allows discovering ServiceMonitors in all namespaces (not just those with `release=prometheus` label)

Note: The ServiceMonitor YAML includes `release: prometheus` label, which helps Prometheus Operator identify it, but with `serviceMonitorSelectorNilUsesHelmValues=false`, it will discover ServiceMonitors in any namespace labeled with `monitoring=enabled`.

Expected output:

```
NAME: prometheus  
LAST DEPLOYED: [timestamp]  
NAMESPACE: monitoring  
STATUS: deployed  
REVISION: 1
```

Label h-game namespace for ServiceMonitor discovery:

```
kubectl label namespace h-game monitoring=enabled
```

Expected output:

```
namespace/h-game labeled
```

Verify installation:

```
kubectl get pods -n monitoring
```

Expected output:

NAME	READY	STATUS	RESTARTS
AGE			
prometheus-prometheus-kube-prometheus-prometheus-0	2/2	Running	0
2m			
prometheus-grafana-7d4b8c9f5-abc12	2/2	Running	0
2m			
prometheus-kube-prometheus-operator-5f8d9c4b6-def34	1/1	Running	0
2m			

Note: You'll see 6-7 pods total, including AlertManager (for alerts), Grafana (dashboards), Prometheus (metrics collection), and node exporters (node metrics).

Step 2: Access Grafana

Time: 10 min

Port-forward Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

⚠ Keep this terminal open! The port-forward must stay running.

Access Grafana:

- Open browser: <http://localhost:3000>
- Username: **admin**
- Password: **admin123**

Tip: Run this in a separate terminal window so you can still use your terminal for other commands.

Expected result:

- Grafana dashboard loads
- Pre-configured dashboards visible (Kubernetes / Compute Resources / Cluster)

Verify Prometheus data source:

1. Go to **Configuration** → **Data Sources**
2. Click "**Prometheus**" (or "Add data source" if not present)
3. **Set URL to:** `http://prometheus-kube-prometheus-prometheus:9090`
 - **Important:** This is the Kubernetes service name (for Grafana running inside K8s)
 - **Don't use:** `http://localhost:9090` (that only works from your browser via port-forward)
 - **Use:** `http://prometheus-kube-prometheus-prometheus:9090` (service name for internal K8s access)
4. Click "**Save & Test**"
5. Should show: "**Data source is working.**"

Understanding the difference:

- **From your browser (via port-forward):** `http://localhost:9090` Works
- **From Grafana (inside Kubernetes):**
`http://prometheus-kube-prometheus-prometheus:9090` Works
- **Why?** Grafana runs inside Kubernetes, so it needs the internal service name, not localhost

Note:

- Grafana is usually pre-configured with the Prometheus data source
- If it's already configured and working, you can skip this step
- If you see connection errors, make sure you're using the service name, not localhost

Step 3: Verify Backend Metrics Endpoint

Time: 5 minutes

What this does: Confirms your backend is already exposing metrics

Your backend already has comprehensive metrics implemented! ([all code files in repo](#))

Let's verify:

Test metrics endpoint:

```
# Port-forward backend service
kubectl port-forward -n h-game svc/backend-service 3001:3001

# In another terminal, test metrics
curl http://localhost:3001/metrics
```

Expected output:

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/health",status_code="200",endpoint_type="health"} 5

# HELP http_request_duration_seconds Duration of HTTP requests in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{method="GET",route="/health",status_code="200",endpoint_type="health",le="0.1"} 5
...

# HELP game_scores_total Total number of game scores submitted
# TYPE game_scores_total counter
...
```

Your backend exposes:

- HTTP metrics (requests, duration, errors)

- Game metrics (scores, completion time, accuracy)
- Database metrics (connections, query duration)
- Redis metrics (connections, cache hit rate)
- System metrics (health, uptime, memory, CPU)

Press Ctrl+C to stop port-forward.

Step 4: Create ServiceMonitor

What this does: Tells Prometheus which service to scrape for metrics

Your services already have named ports! Both `backend-service` and `frontend-service` use `name: http`, which is perfect for ServiceMonitors.

Apply ServiceMonitor:

```
cd "/Users/mac/Desktop/devops playbook/k8s"  
kubectl apply -f servicemonitor.yaml
```

Expected output:

```
servicemonitor.monitoring.coreos.com/h-game-backend created
```

Verify ServiceMonitor:

```
kubectl get servicemonitor -n h-game
```

Expected output:

NAME	AGE
h-game-backend	30s

Verify Service labels match:

```
kubectl get svc backend-service -n h-game -o yaml | grep -A 3  
"labels:"
```

Should show:

```
labels:  
  app: backend
```

This matches the ServiceMonitor selector!

Step 5: Verify Prometheus Discovery

What this does: Confirms Prometheus is scraping your backend

```
Port-forward Prometheus:  
kubectl port-forward -n monitoring  
svc/prometheus-kube-prometheus-prometheus 9090:9090
```

Keep this terminal open!

Access Prometheus:

- Open browser: <http://localhost:9090>
- Go to Status → Targets

Expected result:

- h-game-backend target shows as UP
- Last scrape time is recent (< 1 minute ago)
- No scrape errors

If target doesn't appear:

1. Check ServiceMonitor exists:

```
kubectl get servicemonitor -n h-game
```

Should show: h-game-backend

2. Check Service labels match:

```
kubectl get svc backend-service -n h-game -o yaml | grep -A 3 "labels:"
```

Should show: **app:** backend

3. Check Service has named port:

```
kubectl get svc backend-service -n h-game -o yaml | grep -A 2 "ports:"
```

Should show: **name:** http

4. Check namespace label:

```
kubectl get namespace h-game --show-labels
```

Should show: **monitoring=enabled**

5. Check backend pods are running:

```
kubectl get pods -n h-game -l app=backend
```

Should show **3 pods** in Running state

6. Test metrics endpoint directly:

```
kubectl port-forward -n h-game svc/backend-service 3001:3001  
curl http://localhost:3001/metrics
```

Should return Prometheus metrics

Wait 1-2 minutes: Prometheus discovers targets every 30-60 seconds. Refresh the targets page.

Step 6: Query Metrics with PromQL

What this does: Learn to query your metrics

Access Prometheus UI:

- `http://localhost:9090` (port-forward should still be running)

Tip: If port-forward isn't running, start it:

```
kubectl port-forward -n monitoring  
svc/prometheus-kube-prometheus-prometheus 9090:9090
```

Try these queries:

Request rate (requests per second):

```
rate(http_requests_total[5m])
```

Average response time:

```
rate(http_request_duration_seconds_sum[5m]) /  
rate(http_request_duration_seconds_count[5m])
```

95th percentile latency:

```
histogram_quantile(0.95,  
rate(http_request_duration_seconds_bucket[5m]))
```

Error rate:

```
rate(http_errors_total[5m])
```

Error percentage:

```
rate(http_errors_total[5m]) / rate(http_requests_total[5m]) * 100
```

Game scores by difficulty:

```
sum by (difficulty) (game_scores_total)
```

Database query duration:

```
histogram_quantile(0.95,  
rate(database_query_duration_seconds_bucket[5m]))
```

Redis cache hit rate:

```
redis_cache_hit_rate
```

Expected result:

- Queries return time-series data
- Graphs show metrics over time
- Can identify trends and anomalies

Step 7: Create Grafana Dashboard

What this does: Create custom dashboard for your application metrics

Access Grafana:

- `http://localhost:3000` (port-forward should still be running)
- Login: `admin` / `admin123`

Tip: Keep the port-forward terminal open. If it closes, restart it:

```
kubect1 port-forward -n monitoring svc/prometheus-grafana 3000:80
```

Create new dashboard:

1. Go to Dashboards → New Dashboard
2. Click "Add visualization"
3. Select Prometheus as data source

Add panels:

Panel 1: Request Rate

Query:

```
rate(http_requests_total[5m])
```

- **Legend:** `{{method}}` `{{route}}`

- **Title:** "Request Rate (req/s)"

Panel 2: Request Duration (p95)

Query:

```
histogram_quantile(0.95,  
rate(http_request_duration_seconds_bucket[5m]))
```

- **Legend:** "p95 Latency"
- **Title:** "Response Time (p95)"
- **Unit:** seconds

Panel 3: Error Rate

Query:

```
rate(http_errors_total[5m])
```

- **Legend:** `{{method}}` `{{route}}`
- **Title:** "Error Rate (errors/s)"

Panel 4: Error Percentage

Query:

```
rate(http_errors_total[5m]) / rate(http_requests_total[5m]) * 100
```


- **Title:** "Error Percentage (%)"
- **Unit:** percent

Panel 5: Game Scores by Difficulty

Query:

```
sum by (difficulty) (game_scores_total)
```

- **Title:** "Total Game Scores"
- **Visualization:** Bar chart

Panel 6: Database Query Duration

Query:

```
histogram_quantile(0.95,  
rate(database_query_duration_seconds_bucket[5m]))
```

- **Title:** "Database Query Time (p95)"
- **Unit:** seconds

Panel 7: Redis Cache Hit Rate

Query:

```
redis_cache_hit_rate
```

- **Title:** "Redis Cache Hit Rate"
- **Unit:** percent

Save dashboard:

1. Click "Save dashboard" (top right)
2. Name: "H-Game Metrics"
3. Click "Save"

Expected result:

- Dashboard shows all panels

- Data updates in real-time
- Can see request rates, latency, errors, and business metrics

Step 8: Generate Some Traffic

What this does: Generate traffic to see metrics in action

Get your ALB URL:

```
ALB_URL=$(kubectl get ingress h-game -n h-game -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo "ALB URL: http://$ALB_URL"
```

Generate some requests:

```
# Health check
for i in {1..10}; do curl -s http://$ALB_URL/health > /dev/null; done

# API endpoint
for i in {1..10}; do curl -s http://$ALB_URL/api > /dev/null; done

# Leaderboard
for i in {1..5}; do curl -s http://$ALB_URL/api/leaderboard >
/dev/null; done
```

Watch metrics update:

1. Go to Grafana dashboard
2. Refresh the page
3. You should see metrics increase!

Expected result:

- Request rate increases
- Request duration shows values
- Metrics update in real-time

Troubleshooting

ServiceMonitor not discovered

Check namespace label:

```
kubectl get namespace h-game --show-labels
```

Should show: `monitoring=enabled`

If not, add it:

```
kubectl label namespace h-game monitoring=enabled --overwrite
```

Check ServiceMonitor exists:

```
kubectl get servicemonitor -n h-game
```

Check Service labels:

```
kubectl get svc backend-service -n h-game -o yaml | grep -A 3 "labels:"
```

Must match ServiceMonitor selector:

```
labels:  
  app: backend
```

Prometheus target shows DOWN

Check backend pod is running:

```
kubectl get pods -n h-game -l app=backend
```

Test metrics endpoint directly:

```
kubectl port-forward -n h-game svc/backend-service 3001:3001
```

```
curl http://localhost:3001/metrics
```

Check ServiceMonitor port name:

```
kubectl get servicemonitor h-game-backend -n h-game -o yaml | grep -A 2 "port:"
```

Should show: **port:** http

Check Service port name:

```
kubectl get svc backend-service -n h-game -o yaml | grep -A 2 "ports:"
```

Should show: **name:** http

No metrics in Grafana

Check Prometheus data source:

1. Grafana → Configuration → Data Sources
2. Click "Prometheus"
3. Click "Save & Test"
4. Should show: "Data source is working"

Check queries:

- Make sure metric names match (case-sensitive!)
- Use Prometheus UI to verify metric names exist

Check time range:

- Make sure dashboard time range includes recent data
- Try "Last 5 minutes"

Quick Reference

Access Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80  
# Open: http://localhost:3000 (admin/admin123)
```

Access Prometheus:

```
kubectl port-forward -n monitoring  
svc/prometheus-kube-prometheus-prometheus 9090:9090  
# Open: http://localhost:9090
```

Check ServiceMonitor:

```
kubectl get servicemonitor -n h-game
```

Check Prometheus targets:

- Prometheus UI → Status → Targets

View backend metrics:

```
kubectl port-forward -n h-game svc/backend-service 3001:3001  
curl http://localhost:3001/metrics
```

Success!

You now have:

- Prometheus collecting metrics from your backend
- Grafana dashboards showing system health
- Real-time visibility into performance
- Business metrics (game scores, user engagement)
- Infrastructure metrics (database, Redis)
- ServiceMonitor automatically discovering your backend

Verification Checklist

Before moving on, verify:

- ☐ Prometheus Operator installed (`kubectl get pods -n monitoring`)
- ☐ Grafana accessible (`http://localhost:3000`)
- ☐ ServiceMonitor created (`kubectl get servicemonitor -n h-game`)
- ☐ Namespace labeled (`kubectl get namespace h-game --show-labels`)
- ☐ Backend metrics endpoint working (`curl http://localhost:3001/metrics`)
- ☐ Prometheus target discovered (`http://localhost:9090/targets`)
- ☐ Can query metrics in Prometheus (<http://localhost:9090>)
- ☐ Can view dashboards in Grafana (<http://localhost:3000>)

What's Next?

- **Alerting:** Set up alerts in Prometheus/AlertManager
- **Log Aggregation:** Add Loki for log collection
- **Tracing:** Add Jaeger for distributed tracing
- **Custom Dashboards:** Create more specific dashboards for your use case

Quick Commands Reference

Access Grafana and Prometheus

You need TWO separate terminal windows:

Terminal 1 - Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

- Access: <http://localhost:3000>
- Login: `admin / admin123`
- Keep this terminal open!

Terminal 2 - Prometheus:

```
kubectl port-forward -n monitoring  
svc/prometheus-kube-prometheus-prometheus 9090:9090
```

- Access: <http://localhost:9090>

- Keep this terminal open!

Pro Tip: Use separate terminal windows/tabs so you can keep both port-forwards running while using your terminal for other commands.

Check ServiceMonitor:

```
kubectl get servicemonitor -n h-game
```

Check Prometheus Targets:

- Open <http://localhost:9090/targets>
- Look for `h-game-backend` target (should be UP)

Test Backend Metrics:

```
kubectl port-forward -n h-game svc/backend-service 3001:3001  
curl http://localhost:3001/metrics
```

Key Takeaways

The Big Picture

What you learned in this task:

- Prometheus collects metrics from all services
- Grafana visualizes metrics in dashboards
- ServiceMonitors automatically discover services
- PromQL queries metrics for analysis
- Application instrumentation exposes custom metrics

Metrics collection is now operational!

Critical Concepts

1. Metrics Types:

- Counter: Always increases (total requests)
- Gauge: Can go up or down (CPU usage)
- Histogram: Distribution (request duration)
- Summary: Similar to a histogram with quantiles

2. Prometheus Architecture:

- Pull-based (Prometheus scrapes /metrics endpoints)
- Service discovery (ServiceMonitors)
- Time-series database (optimized for metrics)
- PromQL (powerful query language)

3. ServiceMonitors:

- Tell Prometheus which services to scrape
- Automatically discover services
- Configure scrape intervals and paths

4. Application Instrumentation:

- Expose /metrics endpoint
- Use prom-client (Node.js) or similar libraries
- Track custom business metrics
- Use default metrics (CPU, memory)

AWS Services Monitoring (Optional)

Note: The current setup monitors your Kubernetes pods (frontend, backend) and application metrics. However, your AWS services (RDS PostgreSQL, ElastiCache Redis) are external to Kubernetes and need additional monitoring.

What's Currently Monitored:

- Application-level database metrics (connections, query duration) - via application code
- Application-level Redis metrics (cache hits, operation duration) - via application code
- Kubernetes pod metrics (CPU, memory, network)

What's NOT Directly Monitored:

- RDS instance metrics (CPU, memory, storage, connections from AWS)
- ElastiCache metrics (CPU, memory, network from AWS)
- AWS infrastructure metrics (ALB, EKS cluster health)

To Monitor AWS Services Directly:

Option 1: CloudWatch Exporter (Recommended)

```
# Install CloudWatch Exporter
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
helm install cloudwatch-exporter
prometheus-community/prometheus-cloudwatch-exporter \
  --namespace monitoring \
  --set config.region=us-east-1 \
  --set config.metrics[0].aws_namespace=AWS/RDS \
  --set config.metrics[1].aws_namespace=AWS/ElastiCache
```

Option 2: Application-Level Monitoring (Current) Your application already tracks database and Redis metrics through application code:

- `database_connections_active` - Database connections
- `database_query_duration_seconds` - Query performance
- `redis_cache_hit_rate` - Cache hit rate
- `redis_operation_duration_seconds` - Redis operation time

This is sufficient for most use cases. Direct AWS monitoring is optional and adds CloudWatch costs.

What's Next?

Congratulations! You've implemented metrics collection!

You now have:

- Prometheus collecting metrics from all services
- Grafana dashboards showing system health
- Real-time visibility into performance
- Application-level AWS service metrics (database, Redis)
- Foundation for alerting (O4)

Next: O2 - Log Aggregation (Loki + Promtail)

In O2, you'll learn:

- How to aggregate logs from all pods
- How to search logs with LogQL
- How to integrate logs with Grafana
- How metrics and logs work together

But most importantly: You now understand metrics. Combined with logs (O2) and traces (O3), you'll have complete observability.

O2: Log Aggregation (Loki + Promtail)

Time: 4-5 hours | **Cost:** Free

Mission Brief

Install Loki and Promtail to aggregate logs from all H-Game pods. Configure structured logging and search logs using LogQL queries in Grafana.

Outcomes:

- Understand log aggregation and centralized logging
- Install Loki using Helm
- Install and configure Promtail as a DaemonSet
- Configure structured JSON logging in H-Game
- Search logs using LogQL
- Integrate logs with Grafana dashboards

Prerequisites: O1 (Metrics) complete, Grafana accessible, H-Game deployed

The One Thing That Makes This Click

Log aggregation = searchable library for your logs. Just like a library catalogs all books so you can find any book quickly, Loki catalogs all logs so you can find any log entry quickly across all services.

Without Log Aggregation:

- Logs scattered across pods
- Need to kubectl logs for each pod
- Can't search across services
- Can't correlate events
- Slow troubleshooting

With Log Aggregation:

- All logs in one place
- Search across all services instantly
- Correlate events across pods
- Fast troubleshooting
- Historical log analysis

Simply put: Logs are centralized and searchable, not scattered.

Everything else (LogQL, labels, retention) is just details. The core concept is:

"You can't debug what you can't find."

The Mental Model Shift:

BEFORE O2:

- "Logs are in pod stdout."
- "I'll kubectl logs when something breaks"
- "Can't search across services"

AFTER O2:

- "All logs aggregated in Loki"
- "Search logs instantly with LogQL"
- "Correlate events across all services"

This is the difference between "hunting for logs" and "finding logs instantly."

Now let's see how log aggregation works...

Phase 1: Understanding Log Aggregation (45 min)

What is Log Aggregation?

Log aggregation collects logs from all services into a central location. Instead of checking logs in each pod separately, you query one system to find logs from any service.

The Problem with Pod Logs:

- Logs are ephemeral (lost when the pod restarts)
- Need to check each pod individually
- Can't search across services
- No historical analysis
- Difficult to correlate events

The Solution: Loki

- Aggregates logs from all pods
- Indexed by labels (service, pod, namespace)
- Searchable with LogQL
- Integrated with Grafana
- Lightweight (no full-text indexing)

Why Loki?

Loki is designed for Kubernetes:

- Label based indexing (like Prometheus)
- LogQL query language (similar to PromQL)
- Integrates with Grafana
- Lightweight and efficient
- Open source and free

How Loki Works:

1. Promtail runs as a DaemonSet on each node
2. Promtail scrapes logs from all pods
3. Promtail sends logs to Loki with labels
4. Loki stores logs indexed by labels
5. Grafana queries Loki using LogQL
6. Logs searchable by service, pod, namespace, time

Loki vs. Other Solutions:

- **ELK Stack (Elasticsearch):** Heavy, complex, full-text indexing
- **Loki:** Lightweight, label-based, Kubernetes-native
- **CloudWatch Logs:** AWS-specific, expensive at scale
- **Loki:** Open-source, runs in a cluster, free

Design Exercise 1: What Logs Matter?

Scenario: You need to aggregate logs from H-Game.

Question: Which log fields are most important?

Think about:

- Timestamp (when did it happen?)
- Service name (which service?)
- Log level (error, warn, info)
- Request ID (correlate requests)
- User ID (track user actions)

Your answer: _____

Answer Below 

Answer:

Critical Log Fields:

1. Metadata:

- `timestamp` - When the event occurred
- `service` - Which service (backend, frontend)
- `pod` - Which pod instance
- `namespace` - Which namespace

2. Application Context:

- `level` - Log level (error, warn, info, debug)
- `message` - Log message
- `request_id` - Request identifier (for tracing)
- `user_id` - User identifier (if applicable)

3. Error Context:

- `error` - Error message
- `stack` - Stack trace (for errors)
- `status_code` - HTTP status code

These fields enable powerful log queries and correlation.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is log aggregation?	"Collecting logs from all services into one place."
Why Loki?	"Lightweight, label-based, Kubernetes-native"
How does Loki work?	"Promtail scrapes logs, sends to Loki with labels, Grafana queries."
What logs matter?	"Timestamp, service, level, message, request_id"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's install Loki!

Phase 2: Implementation (3 hours)

Step 0: Install EBS CSI Driver (Required for Loki Persistence)

Why? Loki needs persistent storage (EBS volumes). The EBS CSI driver allows Kubernetes to provision EBS volumes automatically.

Time: 10-15 minutes

0.1: Verify IAM Role Exists

The EBS CSI driver IAM role should already exist in your Terraform configuration. Verify it exists:

```
cd h-game-terraform
terraform state show aws_iam_role.eks_ebs_csi_driver | grep arn
```

Expected: Shows the IAM role ARN (e.g.,
arn:aws:iam::334091769766:role/h-game-learning-eks-ebs-csi-driver-role)

If the role doesn't exist, create it:

```
cd h-game-terraform
terraform apply -target=aws_iam_role.eks_ebs_csi_driver
               -target=aws_iam_role_policy_attachment.eks_ebs_csi_driver_policy
               -auto-approve
```

0.2: Get IAM Role ARN

Get the role ARN (note: role name may vary based on your Terraform `name_prefix`):

```
# Get cluster name
CLUSTER_NAME=$(terraform -chdir=h-game-terraform output -raw eks_cluster_name)

# Get role ARN (try common name patterns)
ROLE_ARN=$(aws iam get-role --role-name h-game-learning-eks-ebs-csi-driver-role --query 'Role.Arn' --output text 2>&1)
```

```
# If not found, check Terraform state for actual role name
if ! echo "$ROLE_ARN" | grep -q "arn:aws:iam"; then
    echo "Role not found. Checking Terraform state..."
    terraform -chdir=h-game-terraform state show
    aws_iam_role.eks_ebs_csi_driver | grep "arn"
    # Use the ARN from the output above
fi

echo "Role ARN: $ROLE_ARN"
```

Expected: Output shows a valid IAM role ARN.

0.3: Install EBS CSI Driver Addon

Install the EBS CSI driver addon to your EKS cluster:

```
# Get cluster name and role ARN
CLUSTER_NAME=$(terraform -chdir=h-game-terraform output -raw
eks_cluster_name)
ROLE_ARN=$(aws iam get-role --role-name
h-game-learning-eks-ebs-csi-driver-role --query 'Role.Arn' --output
text 2>&1)

# Install addon
aws eks create-addon \
    --cluster-name $CLUSTER_NAME \
    --addon-name aws-ebs-csi-driver \
    --service-account-role-arn "$ROLE_ARN" \
    --region us-east-1
```

Expected output:

```
{
  "addon": {
    "addonName": "aws-ebs-csi-driver",
    "clusterName": "h-game-prod",
    "status": "CREATING",
```

```
    ...  
  }  
}
```

o.4: Wait for Addon to be Active

Wait for the addon to become active (takes 2-5 minutes):

```
CLUSTER_NAME=$(terraform -chdir=h-game-terraform output -raw  
eks_cluster_name)  
  
for i in {1..30}; do  
  STATUS=$(aws eks describe-addon \  
    --cluster-name $CLUSTER_NAME \  
    --addon-name aws-ebs-csi-driver \  
    --region us-east-1 \  
    --query 'addon.status' \  
    --output text 2>&1)  
  
  echo "[$i/30] Status: $STATUS"  
  
  if [ "$STATUS" = "ACTIVE" ]; then  
    echo "✅ EBS CSI Driver is ACTIVE!"  
    break  
  elif echo "$STATUS" | grep -q "CREATE_FAILED\|DEGRADED"; then  
    echo "❌ Addon failed! Check status:"  
    aws eks describe-addon \  
      --cluster-name $CLUSTER_NAME \  
      --addon-name aws-ebs-csi-driver \  
      --region us-east-1 \  
      --query 'addon.health.issues' \  
      --output json  
    exit 1  
  fi  
  
  sleep 10  
done
```

Expected: Status changes from **CREATING** → **ACTIVE** (usually within 2-5 minutes).

0.5: Verify EBS CSI Driver Pods

Verify the EBS CSI driver controller and node pods are running:

```
kubectl get pods -n kube-system | grep ebs-csi
```

Expected output:

```
ebs-csi-controller-xxx    6/6    Running    0    2m
ebs-csi-controller-yyy    6/6    Running    0    2m
ebs-csi-node-xxx         3/3    Running    0    2m
ebs-csi-node-yyy         3/3    Running    0    2m
```

Troubleshooting:

- If pods are not running, check addon status: `aws eks describe-addon --cluster-name $CLUSTER_NAME --addon-name aws-ebs-csi-driver --region us-east-1`
- Verify IAM role permissions: `aws iam get-role-policy --role-name h-game-learning-eks-ebs-csi-driver-role --policy-name AmazonEBSCSIDriverPolicy`

Step 1: Install Loki

Time: 15 min

Add Grafana Helm repository:

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Expected output:

```
"grafana" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
Update Complete.
```

Install Loki:

```
helm install loki grafana/loki-stack \
  --namespace monitoring \
  --set loki.persistence.enabled=true \
  --set loki.persistence.size=10Gi \
  --set promtail.enabled=true
```

Expected output:

```
NAME: loki
LAST DEPLOYED: [timestamp]
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
```

Verify installation:

```
kubectl get pods -n monitoring | grep loki
kubectl get pods -n monitoring | grep promtail
kubectl get pvc -n monitoring
```

Expected output:

```
loki-0          1/1   Running   0    3m
loki-promtail-xxx 1/1   Running   0    3m
loki-promtail-yyy 1/1   Running   0    3m
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS	MODES
STORAGECLASS	AGE				
storage-loki-0	Bound	pvc-xxx	10Gi	RWO	gp2
	3m				

Wait for all pods to be running (2-3 minutes).

Troubleshooting:

- **PVC stuck in Pending:** EBS CSI driver may not be active. Re-run Step 0.4.
- **Pod stuck in Pending:** Check PVC status: `kubectl describe pvc storage-loki-0 -n monitoring`
- **Volume binding timeout:** Verify EBS CSI driver pods are running: `kubectl get pods -n kube-system | grep ebs-csi`

Step 2: Configure Loki Data Source in Grafana (5 min)

Port-forward Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

1. Access Grafana:
 - Open: `http://localhost:3000`
 - Login: `admin / admin123`
2. Add Loki data source:
 - Go to Configuration → Data Sources
 - Click "Add data source"
 - Select "Loki"
 - **URL:** `http://loki:3100`

- **Access:** Select "Server (default)" - CRITICAL: This must be "Server", not "Browser"
- **HTTP Method:** GET (default)
- **Max lines:** 1000 (optional, but recommended)
- **Leave all other settings as DEFAULT** (don't add custom queries, derived fields, etc.)
- Click "Save & Test"
- **Should show:** "Data source connected and labels found."

Verification:

- If you see logs in Grafana Explore, the data source is working correctly
- The health check API may show errors, but queries will work if the datasource is properly configured

IMPORTANT: If you see "parse error" or connection fails:

Root Cause: Grafana's health check is sending an invalid LogQL query. This is a configuration issue.

Fix Option 1: Delete and Recreate (Recommended - Easiest)

1. Delete the existing Loki data source:

- Go to Configuration → Data Sources
- Find Loki data source
- Click Delete (bottom of page)

2. Recreate with MINIMAL settings:

- Click "Add data source" → "Loki"
- URL: **http://loki:3100**
- Access: Server (default) ← Critical!
- Leave everything else as DEFAULT (don't add custom queries, derived fields, etc.)
- Click "Save & Test"

Fix Option 2: Check Derived Fields

If Option 1 doesn't work:

1. Go to **Loki data source** → **Derived Fields** section
2. Remove any custom derived fields
3. Save & Test

Fix Option 3: Fix via Grafana API (Automated - Recommended)

This is the fastest way to fix the datasource programmatically:

```
# Get Grafana pod name
GRAFANA_POD=$(kubectl get pods -n monitoring -l
app.kubernetes.io/name=grafana -o
jsonpath='{.items[0].metadata.name}')

# Get admin password (default is usually admin123, or get from
secret)
GRAFANA_PASS=$(kubectl get secret prometheus-grafana -n monitoring -o
jsonpath='{.data.admin-password}' 2>/dev/null | base64 -d || echo
"admin123")

# Get existing Loki datasource UID
LOKI_UID=$(kubectl exec -n monitoring $GRAFANA_POD -c grafana -- curl
-s -u admin:$GRAFANA_PASS "http://localhost:3000/api/datasources"
2>/dev/null | jq -r '[] | select(.type=="loki") | .uid')

# Delete old datasource (if exists)
if [ -n "$LOKI_UID" ] && [ "$LOKI_UID" != "null" ]; then
    echo "Deleting old Loki datasource (UID: $LOKI_UID)..."
    kubectl exec -n monitoring $GRAFANA_POD -c grafana -- curl -s -X
DELETE -u admin:$GRAFANA_PASS
"http://localhost:3000/api/datasources/uid/$LOKI_UID"
fi

# Create new datasource with minimal config
echo "Creating new Loki datasource..."
kubectl exec -n monitoring $GRAFANA_POD -c grafana -- curl -s -X POST
-u admin:$GRAFANA_PASS \
-H "Content-Type: application/json" \
-d
'{"name":"Loki","type":"loki","url":"http://loki:3100","access":"prox
y","isDefault":false}' \
"http://localhost:3000/api/datasources" | jq '.'

# Verify it works
echo "Testing datasource..."
```



```
kubectl exec -n monitoring $GRAFANA_POD -c grafana -- curl -s -u
admin:$GRAFANA_PASS
"http://localhost:3000/api/datasources/proxy/8/loki/api/v1/labels" |
jq '.data | length'
```

Expected output: Should show number of available labels (e.g., **10**).

Fix Option 4: Fix via ConfigMap (If data source was created via Helm)

If the data source is defined in a **ConfigMap**:

```
# Check for datasource ConfigMap
kubectl get configmap -n monitoring | grep datasource

# Edit the ConfigMap (if exists)
kubectl edit configmap prometheus-grafana -n monitoring
```

Ensure Loki datasource has minimal config:

```
datasources:
  datasources.yaml:
    apiVersion: 1
    datasources:
    - name: Loki
      type: loki
      access: proxy
      url: http://loki:3100
      isDefault: false
      editable: true
```

Remove any **jsonData** fields with custom queries, then restart:

```
kubectl rollout restart deployment/prometheus-grafana -n monitoring
```

If still failing:

- Try the fully qualified DNS:
http://loki.monitoring.svc.cluster.local:3100
- Verify Loki is running: **kubectl get pods -n monitoring | grep loki**

- Check Grafana logs: `kubectl logs -n monitoring -l app.kubernetes.io/name=grafana --tail=50 | grep -i loki`
- Test connectivity: `kubectl exec -n monitoring <grafana-pod-name> -c grafana -- curl http://loki:3100/ready`

Troubleshooting:

- Connection failed: Verify Loki service exists: `kubectl get svc -n monitoring | grep loki`
- URL not found: Ensure you're using `http://loki:3100` (not `localhost` or `http://loki:3100`)
- Test connection: `kubectl port-forward -n monitoring svc/loki 3100:3100` then `curl http://localhost:3100/ready`

Step 3: Configure Structured Logging in H-Game

This has been already implemented

Time: 45 min

Backend: Uses Winston for structured JSON logging

Frontend: Uses structured JSON logging utility

What changed:

- All `console.log` → Structured JSON logs with metadata
- Request IDs for correlation
- Log levels (info, warn, error, debug)
- Service names, timestamps, context

Install Winston for structured logging:

```
cd backend
npm install winston
```

Create logger (`backend/logger.js`):

```
const winston = require('winston');
```

```

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: {
    service: 'h-game-backend',
    environment: process.env.NODE_ENV || 'production'
  },
  transports: [
    new winston.transports.Console({
      format: winston.format.json()
    })
  ]
});

module.exports = logger;

```

Update backend server (**backend/server.js**):

```

const express = require('express');
const logger = require('./logger');

const app = express();

// Request ID middleware
app.use((req, res, next) => {
  req.id = require('crypto').randomBytes(16).toString('hex');
  res.setHeader('X-Request-ID', req.id);
  next();
});

// Logging middleware
app.use((req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info('HTTP request', {

```

```

        request_id: req.id,
        method: req.method,
        path: req.path,
        status_code: res.statusCode,
        duration_ms: duration,
        user_agent: req.get('user-agent')
    });
});

    next();
});

// Error logging
app.use((err, req, res, next) => {
    logger.error('Request error', {
        request_id: req.id,
        method: req.method,
        path: req.path,
        error: err.message,
        stack: err.stack
    });

    res.status(500).json({ error: 'Internal server error' });
});

// Existing routes...
app.get('/health', (req, res) => {
    logger.info('Health check', { request_id: req.id });
    res.json({ status: 'ok' });
});

// Start server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    logger.info('Server started', { port: PORT });
});

```

Update frontend logging (`frontend/src/utils/logger.js`):

```

const log = (level, message, metadata = {}) => {
    const logEntry = {
        timestamp: new Date().toISOString(),

```

```

    level,
    message,
    service: 'h-game-frontend',
    ...metadata
  });

  console.log(JSON.stringify(logEntry));
};

export const logger = {
  info: (message, metadata) => log('info', message, metadata),
  warn: (message, metadata) => log('warn', message, metadata),
  error: (message, metadata) => log('error', message, metadata),
  debug: (message, metadata) => log('debug', message, metadata)
};

```

Use logger in frontend:

```

import { logger } from './utils/logger';

// In API calls
const fetchData = async (url) => {
  logger.info('API request', { url });
  try {
    const response = await fetch(url);
    logger.info('API response', { url, status: response.status });
    return response.json();
  } catch (error) {
    logger.error('API error', { url, error: error.message });
    throw error;
  }
};

```

Rebuild and deploy (if not already done):

```

# Get ECR registry
ECR_BACKEND=$(terraform -chdir=h-game-terraform output -raw
ecr_backend_repository_url)
ECR_FRONTEND=$(terraform -chdir=h-game-terraform output -raw
ecr_frontend_repository_url)

```

```
# Build and push backend
cd backend
npm install # Installs winston
docker build -t "${ECR_BACKEND}:latest" .
aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin $(echo $ECR_BACKEND | cut -d '/' -f1)
docker push "${ECR_BACKEND}:latest"

# Build and push frontend
cd ../frontend
docker build -t "${ECR_FRONTEND}:latest" .
aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin $(echo $ECR_FRONTEND | cut -d '/' -f1)
docker push "${ECR_FRONTEND}:latest"

# Update deployments
kubectl rollout restart deployment/backend -n h-game
kubectl rollout restart deployment/frontend -n h-game
```

Step 4: Search Logs with LogQL

Time: 30 min

1. Access Grafana Explore:

```
# Port-forward Grafana (if not already):
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

Note: Keep this running in a terminal or run in background: `kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80 &`

2. **Open browser:** <http://localhost:3000>
3. **Login:** admin / admin123
4. **Go to Explore → Select Loki data source**

Try these LogQL queries:

All logs:

```
{namespace="h-game"}
```

Logs from backend service:

```
{namespace="h-game", app="h-game-backend"}
```

Error logs:

```
{namespace="h-game"} |= "error"
```

Logs with specific request ID:

```
{namespace="h-game"} | json | request_id="abc123"
```

Error rate over time:

```
sum(rate({namespace="h-game"} |= "error" [5m]))
```

Logs by log level:

```
{namespace="h-game"} | json | level="error"
```

HTTP requests:

```
{namespace="h-game"} | json | message="HTTP request"
```

Slow requests (>1 second):

```
{namespace="h-game"} | json | duration_ms > 1000
```

Expected result:

- Queries return matching log entries
- Can filter by labels, text, and JSON fields
- Can aggregate and calculate rates

Tips:

- Use the time range selector (top right) to adjust the time window
- Click on log lines to expand and see full JSON structure
- Use the "Show context" button to see surrounding logs
- Try different label combinations to filter logs

Step 5: Verify Promtail Configuration (15 min)

Note: Promtail is automatically configured by the **loki-stack** Helm chart. It runs as a DaemonSet and automatically discovers pods.

Check Promtail pods are running:

```
kubectl get pods -n monitoring | grep promtail
```

Expected output:

```
loki-promtail-xxx    1/1    Running    0    5m
loki-promtail-yyy    1/1    Running    0    5m
```

Verify Promtail is scraping logs:

```
kubectl logs -n monitoring -l app=promtail --tail=20
```

Expected output:

```
level=info msg="Starting Promtail" version="[version]"
level=info msg="Server started" http_listen_address=0.0.0.0:3101
level=info msg="Target manager started"
level=info msg="Successfully sent batch" ...
```

Check Promtail configuration (if needed):

```
# Promtail config is embedded in the pod, view it:
kubectl exec -n monitoring $(kubectl get pods -n monitoring -l
app=promtail -o jsonpath='{.items[0].metadata.name}') -- cat
/etc/promtail/config.yml
```


Expected: Configuration shows:

- `clients.url`: `http://loki:3100/loki/api/v1/push`
- `kubernetes_sd_configs` for pod discovery
- Label relabeling rules for namespace, pod, app

Troubleshooting:

- If Promtail pods are not running, check: `kubectl describe pod -n monitoring -l app=promtail`
- If no logs are being scraped, verify pods exist: `kubectl get pods -A`
- Check Promtail can reach Loki: `kubectl exec -n monitoring <promtail-pod> -- curl http://loki:3100/ready`

Step 6: Verify Log Aggregation (10 min)

Check Promtail is scraping:

```
kubectl logs -n monitoring -l app=promtail --tail=20
```

Expected: Promtail logs showing it's reading log files and sending to Loki.

Check Loki is receiving logs:

```
# Port-forward Loki
kubectl port-forward -n monitoring svc/loki 3100:3100

# In another terminal, check available namespaces
curl -G -s "http://localhost:3100/loki/api/v1/label/namespace/values"
| jq '.data'
```

Expected: Should show `["h-game", "monitoring", ...]` - your namespaces with logs.

Check application logs are structured:

```
kubectl logs -l app=backend -n h-game --tail=5
```

Expected: JSON formatted logs like:

```
{"timestamp":"2026-02-05T18:57:49.431Z","level":"info","message":"HTTP request","service":"h-game-backend","request_id":"14c55d7bc27817bee6a0ceb45b0e8638","method":"GET","path":"/metrics","status_code":200,"duration_ms":3,"environment":"production"}
```

Verify logs are queryable in Loki:

```
# Query Loki directly (with port-forward running)
curl -G -s "http://localhost:3100/loki/api/v1/query_range" \
  --data-urlencode 'query={namespace="h-game"}' \
  --data-urlencode 'limit=5' \
  --data-urlencode "start=$(date -u -v-5M +%s)000000000" \
  --data-urlencode "end=$(date -u +%s)000000000" | jq '.data.result | length'
```

Expected: Returns number of log entries found (e.g., 5).

Step 7: Create Log Dashboard (30 min)

Create a Grafana dashboard to visualize logs.

Dashboard JSON is already created: grafana/logs-dashboard.json

Import dashboard in Grafana:

1. **Port-forward Grafana** (if not already):

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

2. **Access Grafana:**

- Open: `http://localhost:3000`
- Login: `admin` / `admin123`

3. **Import dashboard:**

- Go to Dashboards → Import
- Click "Upload JSON file"
- Select `grafana/logs-dashboard.json`
- Select Loki as the data source
- Click "Import"

Expected result:

- Dashboard shows 4 panels:
 - **Log Volume:** Total logs per minute
 - **Error Logs:** Real-time error log stream
 - **Logs by Service:** Breakdown by app label
 - **HTTP Request Rate:** Requests per second
- Logs update in real-time (30s refresh)
- Can drill down into specific log entries

Customize the dashboard:

- Click panel title → Edit to modify queries
- Add new panels for specific log patterns
- Adjust time ranges and refresh intervals

- **Save changes:** Save dashboard (top right)

Alternative: Create dashboard manually:

1. Go to Dashboards → New Dashboard
2. Add panel → Select Loki data source
3. Enter LogQL query: `{namespace="h-game"}`
4. Choose visualization type (Logs, Graph, etc.)
5. Repeat for additional panels
6. Save dashboard

Quick Reference

Access Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
# Open: http://localhost:3000 (admin/admin123)
```

Access Loki API:

```
kubectl port-forward -n monitoring svc/loki 3100:3100
# Query: http://localhost:3100
```

View logs in Grafana:

- Go to **Explore** → Select **Loki**
- Enter LogQL query
- Click **Run query**

Common LogQL patterns:

- `{namespace="h-game"}` - All logs
- `{namespace="h-game" } |= "error"` - Error logs
- `{namespace="h-game" } | json | level="error"` - JSON parsed errors
- `sum(rate({namespace="h-game" } |= "error" [5m]))` - Error rate

Troubleshooting

EBS CSI Driver Issues

Addon not active:

1. Check addon status: `aws eks describe-addon --cluster-name $CLUSTER_NAME --addon-name aws-ebs-csi-driver --region us-east-1`
2. Verify IAM role exists: `aws iam get-role --role-name h-game-learning-eks-ebs-csi-driver-role`
3. Check addon health issues: `aws eks describe-addon --cluster-name $CLUSTER_NAME --addon-name aws-ebs-csi-driver --region us-east-1 --query 'addon.health.issues'`

EBS CSI pods not running:

1. Check pods: `kubectl get pods -n kube-system | grep ebs-csi`
2. Check pod logs: `kubectl logs -n kube-system -l app=ebs-csi-controller --tail=50`
3. Verify service account: `kubectl get sa ebs-csi-controller-sa -n kube-system -o yaml`

Loki Issues

Loki pod stuck in Pending:

1. Check PVC status: `kubectl get pvc -n monitoring`
2. Describe PVC: `kubectl describe pvc storage-loki-0 -n monitoring`
3. Verify EBS CSI driver is active (Step 0.4)
4. Check storage class: `kubectl get storageclass`

No logs in Grafana:

1. Check Promtail pods: `kubectl get pods -n monitoring | grep promtail`
2. Check Promtail logs: `kubectl logs -n monitoring -l app=promtail --tail=50`
3. Verify Loki is running: `kubectl get pods -n monitoring | grep loki`

Logs not structured:

1. Verify new images deployed: `kubectl describe deployment/backend -n h-game | grep Image`
2. Check pod logs: `kubectl logs -l app=backend -n h-game --tail=5`

3. Restart pods: `kubectl rollout restart deployment/backend -n h-game`

Loki data source not connecting / Parse errors:

Most common issue: Access mode is set to "Browser" instead of "Server"

Fix:

1. **Delete the existing Loki data source** in Grafana UI
2. **Recreate it** with:
 - URL: `http://loki:3100`
 - Access: **Server (default)** ← Must be this!
 - HTTP Method: GET
3. Click "Save & Test"

Diagnostic steps:

1. Verify Loki service: `kubectl get svc -n monitoring | grep loki`
2. Check URL in Grafana: Should be `http://loki:3100` (same namespace, short name works)
3. **Critical:** Verify "Access" is set to **"Server (default)"** - NOT "Browser"
4. Test connection from Grafana pod:

```
GRAFANA_POD=$(kubectl get pods -n monitoring -l
app.kubernetes.io/name=grafana -o
jsonpath='{.items[0].metadata.name}')
kubectl exec -n monitoring $GRAFANA_POD -c grafana -- curl
http://loki:3100/ready
```

Should return: `ready`

5. Check Grafana logs for parse errors:

```
kubectl logs -n monitoring -l app.kubernetes.io/name=grafana
--tail=50 | grep -i "loki\|parse error"
```

5. If parse errors persist, delete and recreate the data source (see fix above)
6. Check Loki logs: `kubectl logs -n monitoring loki-0 --tail=50`
7. Restart Grafana if needed: `kubectl rollout restart deployment/prometheus-grafana -n monitoring`

Why "Server" mode is required:

- "Browser" mode tries to connect from your browser → Can't reach internal K8s services
- "Server" mode uses Grafana pod → Can reach **loki:3100** via Kubernetes DNS

Step 7: Git Workflow

Time: 10 min

Commit changes:

```
# Add structured logging files
git add backend/logger.js backend/server.js
frontend/src/utlis/logger.js

# Add Grafana dashboard
git add grafana/logs-dashboard.json

# Commit
git commit -m "feat: Add structured logging with Loki aggregation"

- Add Winston logger for backend structured JSON logs
- Add structured logging utility for frontend
- Create Grafana dashboard for log visualization
- Configure Loki data source in Grafana"

# Push to remote
git push origin main
```

Note: If you haven't rebuilt and pushed Docker images yet, do that first:

```
# See Step 3 for full commands
# Build and push backend/frontend images to ECR
# Then update deployments: kubectl rollout restart deployment/backend
-n h-game
```

Expected result:

- Changes committed to Git
- CI/CD pipeline runs (if configured)
- New images deployed with structured logging

Phase 3: Validation (30 min)

Verify Log Aggregation

Check Loki is receiving logs:

```
# Port-forward Loki
kubectl port-forward -n monitoring svc/loki 3100:3100
```

In another terminal, query Loki API:

```
# Get available labels
curl -G -s "http://localhost:3100/loki/api/v1/label/namespace/values" \
| jq '.data'
```

Expected output:

```
["h-game", "monitoring", "kube-system"]
```

Query logs:

```
curl -G -s "http://localhost:3100/loki/api/v1/query_range" \
--data-urlencode 'query={namespace="h-game"}' \
--data-urlencode 'limit=10' \
--data-urlencode "start=$(date -u -v-5M +%s)000000000" \
--data-urlencode "end=$(date -u +%s)000000000" | jq '.data.result | \
length'
```


Expected output: Number of log entries (e.g., 10)

Query specific log stream:

```
curl -G -s "http://localhost:3100/loki/api/v1/query_range" \
  --data-urlencode 'query={namespace="h-game", app="backend"}' \
  --data-urlencode 'limit=5' | jq '.data.result[0].values[0]'
```

Expected output: Shows actual log entry with timestamp and JSON content.

Verify Structured Logging

Check application logs:

```
kubectl logs -l app=backend -n h-game --tail=10
```

Expected output: JSON formatted logs like:

```
{"timestamp":"2026-02-05T18:57:49.431Z","level":"info","message":"HTT
P
request","service":"h-game-backend","request_id":"14c55d7bc27817bee6a
0ceb45b0e8638","method":"GET","path":"/metrics","status_code":200,"du
ration_ms":3,"environment":"production"}
```

Verify logs in Grafana:

1. Port-forward Grafana: `kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80`
2. Open: `http://localhost:3000`
3. Go to **Explore** → Select **Loki**
4. Query: `{namespace="h-game", app="backend"}`
5. Verify JSON logs are parseable
6. Try JSON field queries: `{namespace="h-game"} | json | level="error"`

Expected result:

- Logs are in JSON format
- Can query by JSON fields (level, message, request_id, etc.)
- Logs appear in Grafana Explore
- Dashboard shows log metrics

Knowledge Check

Before moving on, check these boxes:

Understanding Phase:

- ☐ I understand what log aggregation is
- ☐ I know why Loki matters
- ☐ I understand how Promtail works
- ☐ I can explain LogQL basics

Implementation Phase:

- ☐ Loki installed
- ☐ Promtail running
- ☐ Structured logging configured
- ☐ Loki data source added to Grafana
- ☐ LogQL queries working
- ☐ Log dashboard created

Validation Phase:

- ☐ Can query logs in Grafana
- ☐ Can search logs with LogQL
- ☐ Structured logs parseable
- ☐ Logs updating in real-time

Key Takeaways

The Big Picture

What you learned in this task:

- Loki aggregates logs from all pods
- Promtail scrapes logs and sends to Loki
- Structured JSON logging enables powerful queries
- LogQL searches logs by labels and content
- Grafana visualizes logs alongside metrics

Log aggregation is now operational!

Critical Concepts

1. Log Aggregation:

- Centralizes logs from all services
- Enables cross-service search
- Historical log analysis
- Faster troubleshooting

2. Loki Architecture:

- Label-based indexing (like Prometheus)
- Promtail scrapes logs from pods
- Lightweight and efficient
- LogQL query language

3. Structured Logging:

- JSON format for parseable logs
- Consistent fields (timestamp, level, message)
- Enables powerful queries
- Correlates with metrics and traces

4. LogQL:

- Similar to PromQL
- Filter by labels: `{app="backend"}`
- Filter by content: `!= "error"`
- Parse JSON: `| json | level="error"`

- Aggregate: `sum(rate(...))`

What's Next?

Congratulations! You've implemented log aggregation!

You now have:

- All logs aggregated in Loki
- Searchable logs with LogQL
- Structured logging configured
- Log dashboards in Grafana

Next: O3 - Distributed Tracing (Jaeger)

In O3, you'll learn:

- How to trace requests across services
- How to visualize request flows
- How to debug latency issues
- How metrics, logs, and traces work together

But most importantly: You now understand log aggregation. Combined with metrics (O1) and traces (O3), you'll have complete observability.

O3: Distributed Tracing (Jaeger)

Time: 3-4 hours | **Cost:** Free

Mission Brief

Install Jaeger and instrument H-Game with OpenTelemetry to trace requests across services. Visualize request flows and debug latency issues using distributed tracing.

Outcomes:

- Understand distributed tracing and spans
- Install Jaeger using Helm
- Instrument H-Game with OpenTelemetry
- Visualize request flows in Jaeger UI
- Debug latency issues using traces
- Correlate traces with metrics and logs

Prerequisites: O1 (Metrics) and O2 (Logging) complete, H-Game deployed

The One Thing That Makes This Click

Distributed tracing = GPS for requests. Just like GPS shows you the exact route you took and how long each segment took, distributed tracing shows you the exact path a request took through your services and how long each service took.

Without Tracing:

- Don't know which service is slow
- Can't see request flow across services
- Difficult to debug latency issues
- Can't correlate related requests

With Tracing:

- See the exact request path through all services
- Know which service is slow and why
- Debug latency issues instantly
- Correlate requests with metrics and logs

In short: Traces show you the journey of a request through your system.

Everything else (spans, OpenTelemetry, context propagation) is just details. The core concept is:

"You can't optimize what you can't see."

Before O3, slow requests required guessing and checking logs manually.

After O3, requests are traced end-to-end, showing exactly where delays occur.

The shift is from **guessing what's slow** to **knowing what's slow**.

Now let's see how distributed tracing works...

Phase 1: Understanding Distributed Tracing (30 min)

What is Distributed Tracing?

Distributed tracing tracks requests as they flow through multiple services.

Each service creates a "span" representing its part of the request, and spans are connected to form a "trace."

The Problem with Single-Service Logs:

- Request hits multiple services (frontend → backend → database)
- Each service logs independently
- Can't see the full request journey
- Don't know which service is slow
- Difficult to correlate related requests

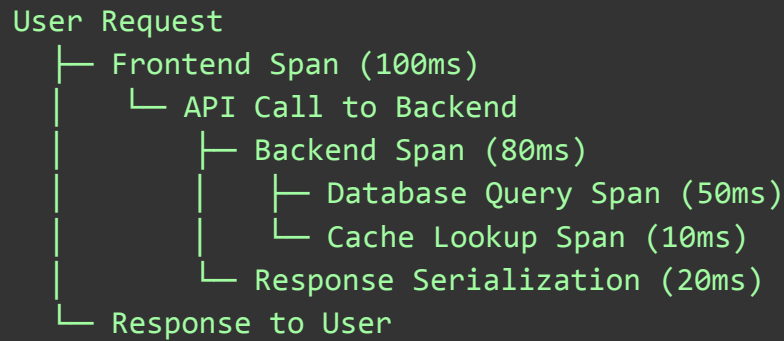
The Solution: Distributed Tracing

- Single trace follows request through all services
- Each service creates a span with timing
- Spans connected to show full request flow
- Visualize request journey in Jaeger UI

Key Concepts:

1. **Trace:** Complete request journey through all services
2. **Span:** Single operation within a service (HTTP handler, database query)
3. **Parent-Child Spans:** Spans connected to show request flow
4. **Context Propagation:** Trace ID passed between services

Trace Example:



Why Jaeger?

Jaeger is the industry standard for distributed tracing:

- Open source and free
- OpenTelemetry compatible
- Powerful UI for visualization
- Supports multiple storage backends
- Integrates with Prometheus and Grafana

How Jaeger Works:

1. Services instrumented with OpenTelemetry
2. OpenTelemetry SDK creates spans
3. Spans sent to the Jaeger collector
4. Jaeger stores traces in the database
5. Jaeger UI visualizes traces
6. Can query traces by service, operation, and duration

Design Exercise 1: What Should Be Traced?

Scenario: You need to trace requests in H-Game.

Question: Which operations should create spans?

Think about:

- HTTP requests (incoming and outgoing)
- Database queries
- External API calls
- Cache operations
- Business logic operations

Your answer: _____

Answer Below 

Answer:

Critical Operations to Trace:

1. HTTP Operations:

- Incoming HTTP requests (frontend, backend)
- Outgoing HTTP requests (backend → external APIs)
- Response times

2. Database Operations:

- Database queries (SELECT, INSERT, UPDATE)
- Query duration
- Connection pool operations

3. External Services:

- API calls to third-party services
- Response times
- Error handling

4. Business Logic:

- Game session creation
- User authentication
- Score calculation

These operations show where time is spent in your system.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is distributed tracing?	"Tracking requests across multiple services"
Why Jaeger?	"Industry standard, OpenTelemetry compatible, powerful UI"
What should be traced?	"HTTP requests, database queries, external API calls"
How does tracing work?	"Services create spans, spans form traces, Jaeger visualizes."

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's install Jaeger!

Phase 2: Implementation (2.5 hours)

Step 1: Install Jaeger

Time: 15 min

Add Jaeger Helm repository:

```
helm repo add jaegertracing
https://jaegertracing.github.io/helm-charts
helm repo update
```

Expected output:

```
"jaegertracing" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
```

Update Complete.

Install Jaeger:

```
helm install jaeger jaegertracing/jaeger \
  --namespace monitoring \
  --set storage.type=memory \
  --set storage.memory.max-traces=50000 \
  --set collector.otlp.enabled=true
```

Note: The `collector.otlp.enabled=true` flag ensures OTLP receivers are enabled on ports 4317 (gRPC) and 4318 (HTTP).

Expected output:

```
NAME: jaeger
LAST DEPLOYED: [timestamp]
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
```

Verify installation:

```
kubectl get pods -n monitoring | grep jaeger
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
jaeger-operator-abc123	1/1	Running	0	2m
jaeger-collector-def456	1/1	Running	0	2m
jaeger-query-ghi789	1/1	Running	0	2m

Note: The Helm chart installs an all-in-one Jaeger deployment by default, which includes collector, query, and storage in a single pod.

Wait for all pods to be running (2-3 minutes).

Step 2: Access Jaeger UI

Time: 10 min

Port-forward Jaeger:

```
kubect1 port-forward -n monitoring svc/jaeger-query 16686:16686
```

Note: The service name is `jaeger` (all-in-one deployment), not `jaeger-query`.

Access Jaeger UI:

- Open browser: `http://localhost:16686`
- Should see Jaeger UI with service selector

Expected result:

- Jaeger UI loads
- Service dropdown shows available services (empty initially)
- Can search for traces

Step 3: Instrument H-Game with OpenTelemetry (All codes Already Implemented, check github Code) [all code files in repo](#)

The following files have been created/updated:

1. `backend/tracing.js` - OpenTelemetry setup
2. `backend/server.js` - Tracing middleware and initialization
3. `backend/logger.js` - Trace ID correlation in logs
4. `backend/middleware/metrics.js` - Trace ID correlation in metrics
5. `k8s/backend-deployment.yaml` - JAEGER_ENDPOINT environment variable

Install OpenTelemetry packages:

```
cd backend  
npm install
```

Note: The implementation uses `@opentelemetry/exporter-otlp-http` (OTLP protocol) instead of the deprecated `@opentelemetry/exporter-jaeger`. This is the modern standard and works with Jaeger's OTLP receiver.

Install OpenTelemetry packages:

```
cd backend
npm install @opentelemetry/api @opentelemetry/sdk-node
@opentelemetry/instrumentation-http
@opentelemetry/instrumentation-express @opentelemetry/exporter-jaeger
```

Create OpenTelemetry setup (`backend/tracing.js`):

```
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { JaegerExporter } = require('@opentelemetry/exporter-jaeger');
const { HttpInstrumentation } =
  require('@opentelemetry/instrumentation-http');
const { ExpressInstrumentation } =
  require('@opentelemetry/instrumentation-express');

const jaegerExporter = new JaegerExporter({
  endpoint: process.env.JAEGER_ENDPOINT ||
    'http://jaeger-collector.monitoring.svc.cluster.local:14268/api/traces',
});

const sdk = new NodeSDK({
  traceExporter: jaegerExporter,
  instrumentations: [
    new HttpInstrumentation(),
    new ExpressInstrumentation(),
  ],
});

sdk.start();

console.log('OpenTelemetry initialized');

module.exports = sdk;
```

Update backend server (**backend/server.js**):

```
// Initialize tracing first
require('./tracing');

const express = require('express');
const { trace, context } = require('@opentelemetry/api');
const logger = require('./logger');

const app = express();

// Tracing middleware
app.use((req, res, next) => {
  const tracer = trace.getTracer('h-game-backend');
  const span = tracer.startSpan('http_request', {
    attributes: {
      'http.method': req.method,
      'http.url': req.url,
      'http.route': req.route ? req.route.path : req.path,
    },
  });

  context.with(trace.setSpan(context.active(), span), () => {
    res.on('finish', () => {
      span.setAttribute('http.status_code', res.statusCode);
      span.end();
    });
    next();
  });
});

// Database query tracing
const traceDatabaseQuery = (query, params) => {
  const tracer = trace.getTracer('h-game-backend');
  const span = tracer.startSpan('database_query', {
    attributes: {
      'db.statement': query,
      'db.params': JSON.stringify(params),
    },
  });

  return span;
};
```

```
// Example: Trace database query
app.get('/api/games', async (req, res) => {
  const span = traceDatabaseQuery('SELECT * FROM games WHERE user_id = $1',
    [req.user.id]);

  try {
    // Database query here
    const games = await db.query('SELECT * FROM games WHERE user_id = $1',
    [req.user.id]);
    span.setAttribute('db.rows_returned', games.length);
    span.end();
    res.json(games);
  } catch (error) {
    span.recordException(error);
    span.setStatus({ code: 2, message: error.message });
    span.end();
    throw error;
  }
});

// Existing routes...
app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

// Start server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  logger.info('Server started', { port: PORT });
});
```

Update frontend (**frontend/src/tracing.js**):

```
import { WebTracerProvider, BatchSpanProcessor } from
 '@opentelemetry/sdk-trace-web';
import { JaegerExporter } from '@opentelemetry/exporter-jaeger';
import { ZoneContextManager } from '@opentelemetry/context-zone';
import { Resource } from '@opentelemetry/resources';
import { SemanticResourceAttributes } from
 '@opentelemetry/semantic-conventions';
```

```

const jaegerExporter = new JaegerExporter({
  endpoint: process.env.REACT_APP_JAEGER_ENDPOINT ||
'http://jaeger-collector.monitoring.svc.cluster.local:14268/api/traces',
});

const provider = new WebTracerProvider({
  resource: new Resource({
    [SemanticResourceAttributes.SERVICE_NAME]: 'h-game-frontend',
  }),
});

provider.addSpanProcessor(new BatchSpanProcessor(jaegerExporter));
provider.register({
  contextManager: new ZoneContextManager(),
});

export const tracer = provider.getTracer('h-game-frontend');

```

Use tracer in frontend:

```

import { tracer } from './tracing';

const fetchData = async (url) => {
  const span = tracer.startSpan('api_request', {
    attributes: {
      'http.method': 'GET',
      'http.url': url,
    },
  });

  try {
    const response = await fetch(url);
    span.setAttribute('http.status_code', response.status);
    span.end();
    return response.json();
  } catch (error) {
    span.recordException(error);
  }
};

```



```
    span.setStatus({ code: 2, message: error.message });
    span.end();
    throw error;
  }
};
```

Update Deployment to set Jaeger endpoint:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: h-game-backend
spec:
  template:
    spec:
      containers:
        - name: backend
          env:
            - name: JAEGER_ENDPOINT
              value:
                "http://jaeger-collector.monitoring.svc.cluster.local:14268/api/traces"
```

Rebuild and push images:

```
# Get ECR registry
ECR_BACKEND=$(terraform -chdir=h-game-terraform output -raw
ecr_backend_repository_url)

# Build backend
cd backend
docker build -t "${ECR_BACKEND}:latest" .

# Login to ECR
aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin $(echo $ECR_BACKEND | cut -d'/' -f1)

# Push image
```

```
docker push "${ECR_BACKEND}:latest"

# Update deployment
kubectl rollout restart deployment/backend -n h-game
```

Expected result:

- Backend image rebuilt with OpenTelemetry
- Image pushed to ECR
- Deployment restarted with new image
- Traces start appearing in Jaeger

Step 4: Generate Traces

Time: 15 min

Make requests to generate traces:

```
# Port-forward backend (in a new terminal or background)
kubectl port-forward svc/backend 3001:3001 -n h-game >
/tmp/backend-pf.log 2>&1 &

# Wait for port-forward to be ready
sleep 2

# Generate traces by making requests
for i in {1..15}; do
    curl -s http://localhost:3001/health > /dev/null
    curl -s http://localhost:3001/api > /dev/null
    echo "Request $i sent"
    sleep 0.5
done

echo " Generated traces"
```

Expected result:

- Requests generate traces

- Traces appear in Jaeger UI

Verify traces are being created (check logs):

```
# Check that trace_id and span_id are in logs
kubectl logs -l app=backend -n h-game --tail=5 | jq '.trace_id,
.span_id'
```

Expected result:

- Requests generate traces
- Logs show `trace_id` and `span_id` fields
- Traces should appear in Jaeger UI (may take 10-30 seconds)

Step 5: Visualize Traces in Jaeger

Time: 30 min

Access Jaeger UI:

```
# Port-forward Jaeger UI (if not already running)
kubectl port-forward -n monitoring svc/jaeger 16686:16686 >
/tmp/jaeger-pf.log 2>&1 &

# Wait for port-forward
sleep 2

# Open browser: http://localhost:16686
```

Verify Jaeger is accessible:

```
# Check services
curl -s http://localhost:16686/api/services | jq '.data'

# Should show: ["jaeger", "h-game-backend"] (after traces are sent)
```

Search for traces in Jaeger UI:

1. **Open browser:** `http://localhost:16686`
2. **Select service:**
 - If `h-game-backend` appears: Select it from the Service dropdown
 - If only `jaeger` appears: Traces haven't reached Jaeger yet. See troubleshooting steps below.
3. **Set time range:** Select "Last 1 hour" or custom range
4. Click "Find Traces"
5. **View traces:** You should see traces listed with:
 - Duration (e.g., "4.2ms")
 - Operation name (e.g., "http_request")
 - Number of spans

Expected result:

- Traces listed with duration
- Can click trace to see span details
- Spans show request flow through services
- Timeline view shows request timing

Analyze a trace:

1. Click on a trace from the list
2. Timeline view shows:
 - All spans in the trace
 - Duration of each span
 - Parent-child relationships
3. **Span details show:**
 - Attributes (http.method, http.url, http.status_code)
 - Duration
 - Start/end times
4. **Identify slow spans:**
 - Red bars indicate slow operations
 - Hover over spans to see details

Expected result:

- The timeline shows the request flow
- Can see which spans are slow
- Can drill down into span details
- Can see HTTP method, URL, status codes

If `h-game-backend` doesn't appear in service dropdown:

This means traces aren't reaching Jaeger yet. Follow these troubleshooting steps:

1. Verify trace IDs are being created:

```
kubectl logs -l app=backend -n h-game --tail=10 | jq '.trace_id' | head -5
```

Should show trace IDs (not null). If null, tracing isn't working.

2. Check OpenTelemetry initialization:

```
kubectl logs -l app=backend -n h-game | grep -i "opentelemetry\|jaeger endpoint"
```

Should show: "OpenTelemetry initialized" and "Jaeger endpoint: ..."

3. Verify endpoint connectivity:

```
BACKEND_POD=$(kubectl get pods -n h-game -l app=backend -o jsonpath='{.items[0].metadata.name}')  
kubectl exec -n h-game $BACKEND_POD -- env | grep JAEGER_ENDPOINT
```

Should show: `http://jaeger.monitoring.svc.cluster.local:4318/v1/traces`

4. Check Jaeger collector is receiving traces:

```
kubectl logs -n monitoring -l app.kubernetes.io/instance=jaeger --tail=50 | grep -i "otlp\|trace\|span\|error"
```

Look for OTLP receiver activity or errors

5. Wait 10-30 seconds: OTLP exporter batches traces before sending. Generate more requests:

```
for i in {1..30}; do curl -s http://localhost:3001/health > /dev/null; sleep 0.5; done
```

6. Check if backend image has latest tracing code:

```
BACKEND_POD=$(kubectl get pods -n h-game -l app=backend -o
jsonpath='{.items[0].metadata.name}')
kubectl exec -n h-game $BACKEND_POD -- test -f /app/tracing.js &&
echo "✅ tracing.js exists" || echo "❌ tracing.js missing"
```

If pods are crashing with missing module errors:

```
# Check pod logs for errors
kubectl logs -n h-game <pod-name> --tail=20

# Common error: "Cannot find module '@opentelemetry/resources'"
# Fix: Add missing packages to package.json:
# - @opentelemetry/resources
# - @opentelemetry/semantic-conventions

# Then rebuild and push image:
cd backend
npm install
docker build -t <ECR_URL>:latest .
docker push <ECR_URL>:latest
kubectl rollout restart deployment/backend -n h-game
```

If deployment rollout is stuck:

```
# Check rollout status
kubectl rollout status deployment/backend -n h-game

# If stuck, force delete old pods
kubectl delete pods -n h-game -l app=backend --force --grace-period=0

# Wait for new pods
kubectl get pods -n h-game -l app=backend -w

# Verify new image is running
kubectl logs -l app=backend -n h-game | grep "Jaeger endpoint"
```

9. If still not working, check for export errors:

- Look for any errors in backend logs related to OpenTelemetry
- Verify the OTLP exporter package is installed: `npm list @opentelemetry/exporter-trace-otlp-http`
- Rebuild and push the image if needed

Step 6: Correlate Traces with Metrics and Logs (20 min)

Already Implemented!

Trace ID in Logs:

- `backend/logger.js` automatically includes `trace_id` and `span_id` in all log entries
- Logs can be correlated with traces using the trace ID

Trace ID in Metrics:

- `backend/middleware/metrics.js` extracts trace context
- Metrics can be correlated with traces via logs (which include both `trace_id` and `metrics`)

Verify Trace Correlation:

```
Check logs include trace_id:
kubectl logs -l app=backend -n h-game --tail=10 | jq
```

Expected output:

```
{
  "timestamp": "2024-01-01T12:00:00.000Z",
  "level": "info",
  "message": "HTTP request",
  "trace_id": "abc123def456",
  "span_id": "789ghi012jkl",
  "request_id": "14c55d7bc27817bee6a0ceb45b0e8638"
}
```

Correlate in Grafana:

1. Port-forward Grafana: `kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80`

2. Open: `http://localhost:3000`
3. Go to Explore → Select Loki
4. Query: `{namespace="h-game", app="backend"} | json | trace_id="abc123def456"`
5. See all logs for that trace

Expected result:

- Logs include `trace_id`
- Can correlate traces with logs and metrics
- Complete observability picture

Step 7: Git Workflow (10 min)

Commit changes:

```
git add backend/tracing.js backend/server.js backend/logger.js
backend/middleware/metrics.js backend/package.json
k8s/backend-deployment.yaml
git commit -m "feat: Add distributed tracing with Jaeger and
OpenTelemetry"

- Install OpenTelemetry packages
- Create tracing.js for OpenTelemetry setup
- Add tracing middleware to server.js
- Include trace_id in logs and metrics
- Add JAEGER_ENDPOINT to deployment"
git push origin main
```

Expected result:

- Changes committed to Git
- CI/CD pipeline runs (if configured)
- New images deployed with tracing

Phase 3: Validation (30 min)

Verify Tracing

Check Jaeger is receiving traces:

```
kubectl logs -n monitoring -l app.kubernetes.io/instance=jaeger --tail=20
```

Expected output:

```
{"level":"info","msg":"Starting HTTP server"}  
{"level":"info","msg":"OTLP receiver started","protocol":"http","port":4318}  
{"level":"info","msg":"Span received","service":"h-game-backend"}
```

Note: Modern Jaeger uses OTLP on ports 4317 (gRPC) and 4318 (HTTP), not the legacy port 14268.

Verify traces in Jaeger UI:

1. Port-forward Jaeger: `kubectl port-forward -n monitoring svc/jaeger 16686:16686`
2. Open `http://localhost:16686`
3. Select service: `h-game-backend`
4. Click "Find Traces"
5. Verify traces appear

Expected result:

- Traces appear in Jaeger UI
- Can see request flow through services
- Spans show timing and attributes

Verify Trace Correlation

```
Check logs include trace_id:  
kubectl logs -l app=backend -n h-game --tail=10 | jq
```

Expected output:

```
{
  "timestamp": "2024-01-01T12:00:00.000Z",
  "level": "info",
  "message": "HTTP request",
  "trace_id": "abc123def456",
  "span_id": "789ghi012jkl"
}
```

Verify metrics correlation:

- Metrics are recorded with request context
- Trace IDs available in logs for correlation
- Can query Prometheus and correlate via logs

Expected result:

- Logs include trace_id
- Metrics include trace context
- Can correlate traces with logs and metrics

Knowledge Check

Before moving on, check these boxes:

Understanding Phase:

- ☐ I understand what distributed tracing is
- ☐ I know why Jaeger matters
- ☐ I understand how OpenTelemetry works
- ☐ I can explain spans and traces

Implementation Phase:

- ☐ Jaeger installed
- ☐ OpenTelemetry configured
- ☐ Application instrumented
- ☐ Traces generating
- ☐ Jaeger UI accessible
- ☐ Trace correlation working

Validation Phase:

- ☐ Can view traces in Jaeger UI
- ☐ Can see request flow through services
- ☐ Can identify slow spans
- ☐ Can correlate traces with logs and metrics

Key Takeaways

The Big Picture

What you learned in this task:

- Distributed tracing tracks requests across services
- OpenTelemetry instruments applications
- Jaeger visualizes traces
- Spans show timing and attributes
- Traces correlate with metrics and logs

Distributed tracing is now operational!

Critical Concepts

1. Distributed Tracing:

- Tracks requests across multiple services
- Each service creates spans
- Spans form traces showing request flow
- Identifies bottlenecks and slow operations

2. OpenTelemetry:

- Industry standard for instrumentation
- Automatic instrumentation for HTTP, Express
- Manual instrumentation for custom operations
- Context propagation between services

3. Jaeger:

- Collects and stores traces
- Powerful UI for visualization
- Query traces by service, operation, duration
- Integrates with Prometheus and Grafana

4. Trace Correlation:

- Add trace_id to logs

- Add trace_id to metrics
- Correlate traces with logs and metrics
- Complete observability picture

Troubleshooting

Jaeger Pods Not Running

Check pod status:

```
kubectl get pods -n monitoring | grep jaeger  
kubectl describe pod -n monitoring <jaeger-pod-name>
```

Check logs:

```
kubectl logs -n monitoring -l app.kubernetes.io/name=jaeger --tail=50
```

Common issues:

- Insufficient resources: Check node capacity
- Image pull errors: Verify image registry access
- Storage issues: Check PVC status for persistent storage

No Traces in Jaeger UI

Verify OpenTelemetry is initialized:

```
kubectl logs -l app=backend -n h-game | grep "OpenTelemetry"
```

Expected: Should see " OpenTelemetry initialized for h-game-backend"

Check Jaeger endpoint:

```
kubectl exec -n h-game <backend-pod> -- env | grep JAEGER_ENDPOINT
```

Expected: Should show

`http://jaeger.monitoring.svc.cluster.local:4318/v1/traces` (OTLP HTTP) or
`http://jaeger.monitoring.svc.cluster.local:4317` (OTLP gRPC)

Test connectivity:

```
# Test OTLP HTTP endpoint (port 4318)
kubectl exec -n h-game <backend-pod> -- curl -v
http://jaeger.monitoring.svc.cluster.local:4318/v1/traces

# Or test OTLP gRPC endpoint (port 4317) - requires grpcurl
kubectl exec -n h-game <backend-pod> -- grpcurl -plaintext
jaeger.monitoring.svc.cluster.local:4317 list
```

Verify traces are being sent:

```
kubectl logs -n monitoring -l app.kubernetes.io/name=jaeger --tail=20
| grep -i "span\|trace\|otlp"
```

Check collector health:

```
# Jaeger health endpoint (port 14269)
JAEGER_POD=$(kubectl get pods -n monitoring -l
app.kubernetes.io/instance=jaeger -o
jsonpath='{.items[0].metadata.name}')
kubectl exec -n monitoring $JAEGER_POD -- curl
http://localhost:14269/
```

Trace ID Not in Logs

Verify logger.js includes trace context:

```
kubectl exec -n h-game <backend-pod> -- cat /app/logger.js | grep
trace
```

Check logs format:

```
kubectl logs -l app=backend -n h-game --tail=5 | jq '.trace_id'
```

Expected: Should show trace IDs (not null)

Note: Frontend tracing is not implemented in this version as it requires a build step for OpenTelemetry browser SDK. For vanilla JS, you can add basic trace context propagation manually if needed, but it's optional for the backend-only tracing setup.

Jaeger UI Not Accessible

Check service:

```
kubectl get svc -n monitoring | grep jaeger
```

Verify port-forward:

```
kubectl port-forward -n monitoring svc/jaeger 16686:16686
```

Test connection:

```
curl http://localhost:16686/api/services
```

Expected: Should return list of services

Quick Reference: Jaeger

Access Jaeger UI:

```
kubectl port-forward -n monitoring svc/jaeger 16686:16686  
# Open: http://localhost:16686
```

Check Jaeger status:

```
kubectl get pods -n monitoring | grep jaeger  
kubectl get svc -n monitoring | grep jaeger
```

View Jaeger logs:

```
kubectl logs -n monitoring -l app.kubernetes.io/instance=jaeger
--tail=50
```

Test OTLP endpoint:

```
# From a backend pod
kubectl exec -n h-game <backend-pod> -- curl -v
http://jaeger.monitoring.svc.cluster.local:4318/v1/traces
```

Common Jaeger queries:

- Service list: `curl http://localhost:16686/api/services`
- Find traces: Use Jaeger UI at `http://localhost:16686`
- Search by service: Select service from dropdown → "Find Traces"
- Query traces via API: `curl "http://localhost:16686/api/traces?service=h-game-backend&limit=10"`

How to Generate and View Traces:

Generate traces:

```
# Port-forward backend (if not already running)
kubectl port-forward svc/backend 3001:3001 -n h-game >
/tmp/backend-pf.log 2>&1 &

# Make requests to generate traces
for i in {1..20}; do
  curl -s http://localhost:3001/health > /dev/null
  curl -s http://localhost:3001/api > /dev/null
  if [ $((i % 5)) -eq 0 ]; then echo "Generated $i requests"; fi
  sleep 0.3
done
echo "✅ Generated 40 total requests"
```

Verify traces in logs:

```
kubectl logs -l app=backend -n h-game --tail=5 | jq '{trace_id, span_id, message}'
```

Should show trace_id and span_id for each request

Wait for traces to be exported (10-30 seconds):

```
sleep 10
```

Verify service appears in Jaeger:

```
curl -s http://localhost:16686/api/services | jq -r '.data[]'
```

Should show: ["jaeger", "h-game-backend"]

1. View in Jaeger UI:

- Open <http://localhost:16686>
- Select service: **h-game-backend** (should now appear in dropdown)
- Click "Find Traces"
- Click on a trace to see details
- View timeline, spans, and attributes

2. Correlate with logs:

- Copy a trace_id from Jaeger
- Search in Loki: `{namespace="h-game"} | json | trace_id="<trace_id>"`
- See all logs for that request

What's Next?

You now have:

- Jaeger installed and running
- OpenTelemetry configured in backend
- Automatic HTTP and Express instrumentation
- Trace ID correlation in logs
- Trace context in metrics
- Complete observability (metrics, logs, traces)

Next: O4 - Alerting (AlertManager + Slack) to get notified when issues occur.

O4: Alerting (AlertManager + Slack)

Time: 3-4 hours | **Cost:** Free

Mission Brief

Configure AlertManager to send alerts to Slack when metrics exceed thresholds. Create alert rules based on SLOs and build runbooks for common issues.

Outcomes:

- Understand alerting principles and best practices
- Configure AlertManager with Slack integration
- Create alert rules for critical metrics
- Test alerting end-to-end
- Build runbooks for common alerts
- Configure alert routing and grouping

Prerequisites: O1 (Metrics) complete, Prometheus installed, Slack workspace available

The One Thing That Makes This Click

Alerting = a smoke detector for your system. Just like a smoke detector alerts you to fire before it spreads, AlertManager alerts you to issues before they impact users.

Without Alerting:

- Find out about issues from users
- No proactive notification
- Issues escalate before detection
- Manual monitoring required

With Alerting:

- Get notified immediately when issues occur
- Proactive problem detection
- Issues caught before user impact
- Automated monitoring

In essence, Alerts notify you of issues before users notice.

Everything else (routing, grouping, runbooks) is just details. The core concept is:

"You can't fix what you don't know is broken."

The Mental Model Shift:

BEFORE O4:

- "I'll check metrics when I remember"
- "Users report issues first"
- "Reactive troubleshooting"

AFTER O4:

- "Alerts notify me immediately"
- "Issues detected before users notice."
- "Proactive problem resolution"

This is the difference between "reactive" and "proactive."

Now let's see how alerting works...

Phase 1: Understanding Alerting (30 min)

What is Alerting?

Alerting notifies you when metrics exceed thresholds or conditions are met. Instead of manually checking metrics, AlertManager monitors metrics continuously and sends notifications when issues occur.

The Problem with Manual Monitoring:

- Need to check metrics regularly
- Easy to miss issues
- No notification when problems occur
- Issues discovered by users first

The Solution: AlertManager

- Continuously monitors metrics
- Evaluates alert rules

- Sends notifications when alerts fire
- Groups related alerts
- Routes alerts to appropriate channels

Alert Lifecycle:

1. **Alert Rule:** Defines condition (e.g., error rate > 1%)
2. **Alert Fires:** Condition met, alert becomes active
3. **Notification:** AlertManager sends notification
4. **Alert Resolves:** Condition no longer met, alert becomes resolved
5. **Resolve Notification:** AlertManager sends resolve notification

Alert States:

- **Pending:** Condition met but not yet firing (waiting for duration)
- **Firing:** Alert is active and notifications sent
- **Resolved:** Condition no longer met, alert resolved

Why AlertManager?

AlertManager is the standard for Prometheus alerting:

- Integrated with Prometheus
- Powerful routing and grouping
- Supports multiple notification channels (Slack, PagerDuty, email)
- Handles alert deduplication
- Open-source and free

How AlertManager Works:

1. Prometheus evaluates alert rules
2. When condition met, Prometheus sends alert to AlertManager
3. AlertManager routes alert based on labels
4. AlertManager groups related alerts
5. AlertManager sends notification to Slack
6. When condition resolves, AlertManager sends resolve notification

Design Exercise 1: What Should Alert?

Scenario: You need to configure alerts for H-Game.

Question: Which metrics should trigger alerts?

Think about:

- Error rate (too many errors)
- Latency (requests too slow)
- Availability (service down)
- Resource usage (CPU, memory high)
- Business metrics (active users dropping)

Your answer: _____

Answer Below 

Answer:

Critical Alerts:

1. Availability:

- Service down (no requests for 5 minutes)
- Health check failures
- Pod crashes

2. Error Rate:

- Error rate > 1% for 5 minutes
- 5xx errors > 10 in 5 minutes
- Database connection errors

3. Latency:

- p95 latency > 1 second for 5 minutes
- p99 latency > 3 seconds for 5 minutes
- Database query time > 500ms

4. Resource Usage:

- CPU usage > 80% for 10 minutes
- Memory usage > 90% for 10 minutes
- Disk usage > 85%

5. Business Metrics:

- Active users dropped > 50% in 1 hour
- Request rate dropped > 50% in 1 hour

These alerts catch issues before they impact users.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is alerting?	"Notifying when metrics exceed thresholds."
Why AlertManager?	"Integrated with Prometheus, powerful routing, multiple channels."
What should alert?	"Error rate, latency, availability, resource usage"
How does alerting work?	"Prometheus evaluates rules, AlertManager routes notifications."

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's configure AlertManager!

Phase 2: Implementation (2.5 hours)

Step 1: Configure Slack Webhook

Create Slack App:

1. Go to <https://api.slack.com/apps>
2. Click "Create New App"
3. Choose "From scratch."
4. Name: "H-Game Alerts"
5. Workspace: Select your workspace
6. Click "Create App"

Configure Incoming Webhook:

1. Go to "Incoming Webhooks"
2. Toggle "Activate Incoming Webhooks" to ON
3. Click "Add New Webhook to Workspace"
4. Select channel: `#alerts` (or create new channel)
5. Click "Allow"
6. Copy webhook URL

Expected result:

- **Webhook URL like:**
[https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXX
XXXXXXXXXXXXX](https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXX
XXXXXXXXXXXXX)

Store webhook URL as Kubernetes Secret:

```
kubectl create secret generic slack-webhook \
--from-literal=url='https://hooks.slack.com/services/YOUR/WEBHOOK/URL' \
-n monitoring
```

Expected output:

```
secret/slack-webhook created
```

Step 2: Configure AlertManager

Time: 30 min

AlertManager Config Created: [k8s/alertmanager-config.yaml](#)

Method: Update the existing AlertManager secret

The Prometheus Operator manages AlertManager and uses a specific secret. We need to update that secret:

Step 1: Get your Slack webhook URL

1. Create Slack webhook (see Step 1)
2. Get the URL from the secret:

```
kubectl get secret slack-webhook -n monitoring -o  
jsonpath='{.data.url}' | base64 -d
```

3. Copy the webhook URL

Step 2: Edit AlertManager config file

1. Open `k8s/alertmanager-config.yaml`
2. Find the line: `slack_api_url: 'SLACK_WEBHOOK_URL'`
3. Replace `SLACK_WEBHOOK_URL` with your actual webhook URL
4. Save the file

Step 3: Apply the updated config

```
kubectl apply -f k8s/alertmanager-config.yaml
```

Expected output:

```
secret/alertmanager-prometheus-kube-prometheus-alertmanager  
configured
```


Step 4: Restart AlertManager to reload config

```
# Delete the AlertManager pod to force reload
kubectl delete pod -n monitoring -l
app.kubernetes.io/name=alertmanager

# Wait for pod to restart (watch status)
kubectl get pods -n monitoring | grep alertmanager
```

Wait for pod to be Running:

```
# Watch until pod is ready
kubectl get pods -n monitoring -l app.kubernetes.io/name=alertmanager
-w
# Press Ctrl+C when pod shows "2/2 Running"
```

Expected output:

```
alertmanager-prometheus-kube-prometheus-alertmanager-0  2/2
Running    0    1m
```

Step 5: Verify AlertManager config

```
# Check if config was loaded
kubectl get secret
alertmanager-prometheus-kube-prometheus-alertmanager -n monitoring -o
jsonpath='{.data.alertmanager\.yaml}' | base64 -d | grep -i slack
```

Expected: Should show your Slack webhook URL (not "SLACK_WEBHOOK_URL")

Step 6: Verify AlertManager UI

```
# Port-forward AlertManager
kubectl port-forward -n monitoring
svc/prometheus-kube-prometheus-alertmanager 9093:9093
```

Open browser: `http://localhost:9093`

Expected result:

- AlertManager UI loads
- Can see configuration
- Can see active alerts (if any)

Alternative: Use AlertmanagerConfig CR (if secret method doesn't work)

If the secret method doesn't work, you can use AlertmanagerConfig CR:

```
apiVersion: monitoring.coreos.com/v1
kind: AlertmanagerConfig
metadata:
  name: h-game-alertmanager-config
  namespace: monitoring
spec:
  route:
    receiver: 'slack-notifications'
    groupBy: ['alertname']
  receivers:
  - name: 'slack-notifications'
    slackConfigs:
    - apiURL:
        key: url
        name: slack-webhook
      channel: '#alerts'
      sendResolved: true
```

Then apply:

```
kubectl apply -f k8s/alertmanager-config-cr.yaml # (create this file
if needed)
```

Step 3: Create Alert Rules (45 min)

Alert Rules Created: `k8s/prometheus-rules.yaml`

Apply alert rules:

```
kubectl apply -f k8s/prometheus-rules.yaml
```

Expected output:

```
prometheusrule.monitoring.coreos.com/h-game-alerts created
```

Verify alert rules:

```
kubectl get prometheusrule -n monitoring | grep h-game
```

Expected output:

```
h-game-alerts      30s
```

Verify rules are loaded in Prometheus:

```
# Port-forward Prometheus (if not already running)
kubectl port-forward -n monitoring
svc/prometheus-kube-prometheus-prometheus 9090:9090 >
/tmp/prometheus-pf.log 2>&1 &
# Wait a few seconds
sleep 5
# Check if rules are loaded
curl -s "http://localhost:9090/api/v1/rules?type=alert" | jq -r
'.data.groups[] | select(.name | contains("h-game")) | "\(.name):
\(.rules | length) rules"'
```

Expected output:

```
h-game.rules: 7 rules
```

Check alerts in Prometheus UI:

```
Port-forward Prometheus:
kubectl port-forward -n monitoring
svc/prometheus-kube-prometheus-prometheus 9090:9090
```

1. Open `http://localhost:9090`
2. Go to **Alerts** tab
3. Search for "h-game" or "ServiceDown"
4. Verify alert rules appear

Expected result:

- Alert rules listed (ServiceDown, HighErrorRate, HighLatency, etc.)
- Alerts show as "Inactive" (conditions not met)
- Can see alert expressions and thresholds
- Can see "for" duration and labels

Step 4: Test Alerting (30 min)

Test ServiceDown Alert:

Step 1: Scale down backend

```
kubectl scale deployment backend --replicas=0 -n h-game
```

Step 2: Monitor alert in Prometheus

```
# Port-forward Prometheus (if not already running)
kubectl port-forward -n monitoring
svc/prometheus-kube-prometheus-prometheus 9090:9090

# Open: http://localhost:9090/alerts
# Look for "ServiceDown" alert
# It will show "Pending" for 5 minutes, then "Firing"
```

Step 3: Wait for alert to fire (5 minutes)

- Alert will be "Pending" for 5 minutes (the "for" duration)
- After 5 minutes, alert will change to "Firing"
- Check Slack `#alerts` channel for notification

Expected result:

- Alert fires after 5 minutes
- Slack notification sent to `#alerts` channel
- Alert shows service is down

- Notification includes alert name and description

Step 4: Restore service

```
kubectl scale deployment backend --replicas=3 -n h-game
```

Step 5: Wait for alert to resolve (1-2 minutes)

- Alert will change from "Firing" to "Resolved"
- Check Slack for resolve notification

Expected result:

- Alert resolves when service restored
- Slack notification sent showing alert resolved
- Notification indicates alert is now resolved

Test Error Rate Alert:

Step 1: Port-forward backend

```
kubectl port-forward svc/backend 3001:3001 -n h-game > /tmp/backend-pf.log 2>&1 &
```

Step 2: Generate errors

```
# Generate 100 requests to non-existent endpoint
for i in {1..100}; do
  curl -s http://localhost:3001/nonexistent > /dev/null
  sleep 0.1
done
echo "✅ Generated 100 error requests"
```

Step 3: Monitor error rate

```
# Check error rate in Prometheus
# Query: rate(http_errors_total{job="backend-service"}[5m]) /
rate(http_requests_total{job="backend-service"}[5m])
```

Step 4: Check for alert

- If error rate > 1% for 5 minutes, alert will fire
- Check Prometheus Alerts page
- Check Slack for notification

Expected result:

- Error rate increases
- Alert fires if error rate > 1% for 5 minutes
- Slack notification sent with error rate details

Step 5: Create Runbooks (30 min)

Runbooks Created:

- `runbooks/runbook-template.md` - Template for new runbooks
- `runbooks/service-down.md` - Runbook for ServiceDown alert
- `runbooks/high-error-rate.md` - Runbook for HighErrorRate alert

Link runbooks in Slack alerts:

Update AlertManager config to include runbook links in annotations (already in `k8s/alertmanager-config.yaml`).

Create additional runbooks as needed:

- HighLatency
- VeryHighLatency
- HighCPUUsage
- HighMemoryUsage
- LowRequestRate

Step 6: Git Workflow (10 min)

Commit changes:

```
git add k8s/alertmanager-config.yaml k8s/prometheus-rules.yaml
runbooks/
git commit -m "feat: Add AlertManager with Slack integration and
alert rules"
```

```
- Configure AlertManager with Slack webhook
- Create alert rules for critical metrics
- Add runbooks for common alerts
- Fix job names to use backend-service"
git push origin main
```

Expected result:

- Changes committed to Git
- Alert rules and runbooks are documented

Phase 3: Validation (30 min)

Verify AlertManager Configuration

Check AlertManager config:

```
kubectl get secret
alertmanager-prometheus-kube-prometheus-alertmanager -n monitoring -o
jsonpath='{.data.alertmanager\.yaml}' | base64 -d
```

Expected output:

- AlertManager config shows Slack webhook URL
- Routes configured correctly
- Receivers configured

Access AlertManager UI:

```
kubectl port-forward -n monitoring
svc/prometheus-kube-prometheus-alertmanager 9093:9093
```

Open browser: <http://localhost:9093>

Expected result:

- AlertManager UI loads
- Can see active alerts

- Can see alert history

Verify Alert Rules

Check alert rules in Prometheus:

1. Port-forward Prometheus: `kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090`
2. Open `http://localhost:9090`
3. Go to **Alerts**
4. Verify that all alert rules appear

Expected result:

- All alert rules are listed
- Alert expressions correct
- Thresholds appropriate

Verify Slack Integration

Test alert in Slack:

1. Trigger test alert (scale down service)
2. Wait for alert to fire
3. Check Slack `#alerts` channel

Expected result:

- Alert notification appears in Slack
- Alert shows correct information
- Resolve notification appears when alert resolves

Key Takeaways

Critical Concepts

1. **Alerting Principles:**
 - Alert on symptoms, not causes
 - Alert on user impact
 - Avoid alert fatigue
 - Use appropriate severity levels
 - Include actionable information

2. Alert Lifecycle:

- Pending: Condition met, waiting for duration
- Firing: Alert active, notifications sent
- Resolved: Condition no longer met

3. Alert Rules:

- Define conditions using PromQL
- Set appropriate thresholds
- Use "for" duration to avoid noise
- Include meaningful annotations

4. Runbooks:

- Document investigation steps
- Provide resolution steps
- Include prevention measures
- Link from alert notifications

Next: O5 - Dashboards & SLOs to define reliability targets and create unified dashboards.

O5: Dashboards & SLOs

Time: 3-4 hours | **Cost:** Free

Mission Brief

Define Service Level Objectives (SLOs) for H-Game and create unified dashboards that combine metrics, logs, and traces. Build on-call dashboards and track error budgets.

Outcomes

- Understand SLOs, SLIs, and error budgets
- Define SLOs for H-Game services
- Create unified observability dashboards
- Track error budgets in Grafana
- Build on-call dashboards for incident response
- Integrate SLOs with alerting

Prerequisites

- O1 (Metrics) complete
- O2 (Logging) complete
- O3 (Tracing) complete
- O4 (Alerting) complete

The One Thing That Makes This Click

SLOs = reliability contracts with users. Just like a service guarantee promises 99.9% uptime, SLOs promise specific reliability levels (e.g., 99.9% availability, p95 latency < 1s).

Without SLOs:

- No clear reliability goals
- Can't measure if service is reliable enough
- Don't know when to stop optimizing
- Reactive reliability management

With SLOs:

- Clear reliability targets

- Measurable success criteria
- Error budgets guide decisions
- Proactive reliability management

Simply put: SLOs define how reliable your service should be, and error budgets tell you when to stop optimizing.

Everything else (SLIs, error budgets, dashboards) is just details. The core concept is:

"You can't manage what you don't measure, and you can't improve what you don't target."

The Mental Model Shift:

- **BEFORE O5:** "Service should be reliable" → "I'll optimize when I have time" → "No clear reliability goals"
- **AFTER O5:** "SLO: 99.9% availability, p95 < 1s" → "Error budget: 43 minutes/month" → "Data-driven reliability decisions"

This is the difference between "**hope it's reliable**" and "**know it's reliable.**"

Phase 1: Understanding SLOs (30 min)

What are SLOs?

Service Level Objectives (SLOs) define reliability targets for your service. They specify how reliable your service should be (e.g., 99.9% availability, p95 latency < 1 second).

Key Concepts

- **SLI (Service Level Indicator):** What you measure (availability, latency, error rate)
- **SLO (Service Level Objective):** Target for SLI (99.9% availability)
- **Error Budget:** Amount of unreliability allowed (0.1% = 43 minutes/month)

Example SLOs for H-Game

- **Availability:** 99.9% (43 minutes of downtime/month)
- **Latency:** p95 < 1 second, p99 < 3 seconds
- **Error Rate:** < 0.1% (1 error per 1000 requests)

Why SLOs Matter

- Define clear reliability goals
- Enable data-driven decisions
- Balance reliability and innovation
- Guide when to optimize vs. when to ship features

Error Budget Concept

If your SLO is 99.9% availability:

- **Error Budget:** 0.1% = 43 minutes/month
- **Spend budget on:** Planned maintenance, deployments, experiments
- **When budget exhausted:** Stop new features, focus on reliability

Design Exercise: Define SLOs

Appropriate SLOs for H-Game:

1. **Availability:**
 - Target: 99.9% (43 minutes/month downtime)
 - Reasonable for gaming service
 - Allows for deployments and maintenance
2. **Latency:**
 - p95 < 1 second (95% of requests < 1s)
 - p99 < 3 seconds (99% of requests < 3s)
 - Good user experience for gaming
3. **Error Rate:**
 - < 0.1% (1 error per 1000 requests)
 - Low enough for good user experience
 - Achievable with proper error handling

These SLOs balance user experience with technical feasibility.

Phase 2: Implementation (2.5 hours)

Step 1: Define SLOs for H-Game (20 min)

SLO Definitions Created:

- `slo/slo-definitions.yaml` - SLO definitions
- `docs/slos.md` - SLO documentation

SLO Definitions:

1. **Availability:** 99.9% (43 minutes/month)
2. **Latency p95:** < 1 second
3. **Latency p99:** < 3 seconds
4. **Error Rate:** < 0.1%

Step 2: Create SLO Recording Rules (30 min)

SLO Recording Rules Created: `k8s/slo-recording-rules.yaml`

Apply recording rules:

```
kubectl apply -f k8s/slo-recording-rules.yaml
```

Expected output:

```
prometheusrule.monitoring.coreos.com/h-game-slos created
```

Verify recording rules:

```
kubectl get prometheusrule -n monitoring
```

Expected output:

NAME	AGE
h-game-alerts	1h
h-game-slos	30s

Step 3: Create Unified Observability Dashboard (45 min)

Unified Dashboard Created: `grafana/unified-dashboard.json`

Import dashboard in Grafana:

Port-forward Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

1. Open: `http://localhost:3000`
2. Login: `admin / admin123`

3. Go to **Dashboards** → **Import**
4. Click **"Upload JSON file"**
5. Select `grafana/unified-dashboard.json`
6. Select **Prometheus** and **Loki** as data sources
7. Click **"Import"**

Expected result:

- Unified dashboard shows metrics, logs, and traces
- SLO status and error budgets visible
- All observability data in one place

Step 4: Create On-Call Dashboard (30 min)

On-Call Dashboard Created: `grafana/oncall-dashboard.json`

Import on-call dashboard:

Step 1: Access Grafana Import

- Go to **Dashboards** → **Import**
- (Grafana should still be port-forwarded from Step 3)

Step 2: Upload dashboard

1. Click **"Upload JSON file"** button
2. Navigate to and select: `grafana/oncall-dashboard.json`
3. Click **"Load"**

Step 3: Configure data sources

1. Select **Prometheus** for Prometheus queries
2. Select **Loki** for Loki queries
3. Click **"Import"**

Expected result:

- Dashboard imported successfully
- Shows 5 panels:
 - Critical Alerts (table)
 - Service Status (stat)
 - Error Budget Remaining (gauge)
 - Current Latency (stat)
 - Recent Errors (logs)

- All panels show critical information
- Quick access to logs and metrics
- Optimized for incident response

Step 4: Set as favorite

- Click star icon (top right) to favorite dashboard
- Makes it easy to find during incidents

Step 5: Create SLO Alert Rules (20 min)

SLO Alert Rules Created: `k8s/slo-alert-rules.yaml`

Apply SLO alert rules:

```
kubectl apply -f k8s/slo-alert-rules.yaml
```

Expected output:

```
prometheusrule.monitoring.coreos.com/h-game-slo-alerts created
```

Verify SLO alert rules:

```
kubectl get prometheusrule -n monitoring | grep h-game
```

Expected output:

```
h-game-alerts      1h
h-game-slo-alerts  30s
h-game-slos        1h
```

Check SLO alerts in Prometheus:

1. Port-forward Prometheus: `kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090`
2. Open `http://localhost:9090`
3. Go to **Alerts** tab
4. Search for "SLO" alerts

Expected result:

- SLO alert rules listed:
 - SLOAvailabilityViolation
 - SLOLatencyViolation
 - SLOErrorRateViolation
- Alerts show as "Inactive" (if SLOs met) or "Pending" (if error budgets at risk)
- Alerts fire when error budgets consumed > 50%

Step 6: Git Workflow (10 min)

Commit changes:

```
git add slo/ docs/slos.md k8s/slo-recording-rules.yaml
k8s/slo-alert-rules.yaml grafana/unified-dashboard.json
grafana/oncall-dashboard.json
git commit -m "feat: Add SLOs, error budgets, and unified dashboards"

- Define SLOs for availability, latency, and error rate
- Create SLO recording rules
- Create unified observability dashboard
- Create on-call dashboard
- Add SLO alert rules"
git push origin main
```

Expected result:

- Changes committed to Git
- SLOs documented
- Dashboards and rules versioned

Phase 3: Validation (30 min)

Verify SLO Recording Rules

Check SLO metrics in Prometheus:

1. Port-forward Prometheus: `kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090`
2. Open `http://localhost:9090`

3. Query: `slo:availability:ratio`
4. Verify metric exists and has values

Expected result:

- SLO metrics exist
- Metrics have current values
- Error budget calculations correct

Verify Dashboards

Step 1: Access Grafana

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

Step 2: Open Grafana

- Open: `http://localhost:3000`
- Login: `admin / admin123`

Step 3: Check unified dashboard

1. Go to **Dashboards** (left sidebar)
2. Find "**H-Game Unified Observability**"
3. Click to open dashboard
4. Verify all panels show data:
 - **SLO Status:** Shows availability, latency, error rate
 - **Error Budget:** Shows error budget consumption
 - **Request Rate:** Graph of requests over time
 - **Latency:** p50, p95, p99 latency graphs
 - **Error Rate:** Error rate over time
 - **Recent Logs:** Log stream from backend

Expected result:

- Dashboard loads successfully
- All panels render (may show "No data" if no metrics yet)
- SLO status visible
- Error budgets displayed
- Metrics, logs integrated
- Can adjust time range and refresh

Step 4: Check on-call dashboard

1. Go to **Dashboards**
2. Find "**H-Game On-Call Dashboard**"
3. Click to open dashboard
4. Verify critical information:
 - **Critical Alerts:** Table of firing alerts
 - **Service Status:** UP/DOWN status
 - **Error Budget Remaining:** Gauge showing budget
 - **Current Latency:** p95 and p99 latency
 - **Recent Errors:** Error logs stream

Expected result:

- Dashboard loads successfully
- Alerts visible (if any firing)
- Service status shown
- Error budgets displayed
- Quick access to logs
- Optimized for incident response

Verify SLO Alerts

Check SLO alerts in Prometheus:

1. Go to **Alerts**
2. Verify SLO alert rules appear
3. Check alert states

Expected result:

- SLO alert rules listed
- Alerts show as "Inactive" (if SLOs met)
- Alerts fire when error budgets consumed

Key Takeaways

Critical Concepts

1. **SLOs:**
 - Define reliability targets (availability, latency, error rate)
 - Measurable and achievable
 - Balance user expectations with technical constraints
 - Guide reliability decisions

2. Error Budgets:

- Amount of unreliability allowed
- Spend on deployments, experiments, maintenance
- When exhausted, focus on reliability
- Enable data-driven decisions

3. Unified Dashboards:

- Combine metrics, logs, and traces
- Show SLO status and error budgets
- Provide complete observability picture
- Enable fast troubleshooting

4. On-Call Dashboards:

- Critical information at a glance
- Alerts, service status, error budgets
- Quick access to logs and metrics
- Optimized for incident response

What's Next?

You now have:

- Complete observability stack (metrics, logs, traces)
- Alerting configured with Slack
- SLOs defined and tracked
- Unified and on-call dashboards
- Production-grade observability

Next: Section 9 - GitOps (ArgoCD) to automate deployments with GitOps principles.